

Highlights

Deployment & Evaluation of Modern ROS2 Navigation Algorithms for High Autonomy in Complex Environments

Tomas Horelican, Ludek Zalud

- Full-scale real-world navigation task evaluation of *Navigation2* planning algorithms.
- Unified configuration and deployment demonstration of the *Navigation2* framework.
- Reactive algorithms perform well in unfavorable situations even with simple localization/perception.

Deployment & Evaluation of Modern ROS2 Navigation Algorithms for High Autonomy in Complex Environments

Tomas Horelican, Ludek Zalud

Central European Institute of Technology, Brno University of Technology, Brno, 61200, Czech Republic

Department of Control and Instrumentation, Faculty of Electrical Engineering and Communication, Brno University of Technology, Brno, 61600, Czech Republic

Abstract

The next-generation *Robot Operating System* (ROS2) framework has been gaining wider adoption among research as well as commercial applications thanks to its high modularity and attention to performance, safety, and deterministic behavior. The open-source community-driven *Navigation2* project provides various latest higher-level algorithm implementations realized on top of the ROS2 foundation, relating to localization, mapping, and navigation. Although these algorithms have been thoroughly evaluated individually, a full solution integration still warrants more testing. This paper will assess the effectiveness and reliability of several of the available algorithms in complete navigation task scenarios in terms of times to reach the desired goal, planning and navigation failures, dynamic obstacle avoidance handling, and goal reaching precision. Configuration and deployment steps will be outlined for reproducibility. Directions for future extension options will also be mentioned based on the wider state-of-the-art situation in the field. All tests were performed using a custom-developed differential-drive robotic platform, where the *Navigation2* features have been seamlessly integrated into our system.

Keywords:

autonomous navigation, ROS2, Navigation2, mobile robot, complex environment, localization, SLAM, planning

1. Introduction

New developments in recent years have spurred an increased interest in mobile robotic platforms for commercial and industrial usage as their scope of capabilities rapidly grows. Most notably, for autonomous area surveillance tasks, increased factory production autonomy, search-and-rescue missions after disaster events, or exploration of potentially dangerous environments [1, 2, 3, 4]. While manually operated robot solutions provide a safer extension to the operator's potential abilities, a need for higher autonomy is desired, especially in extreme communication-limited situations or to achieve greater user convenience. Environments with limited access to external sources of localization (*Global Navigation Satellite System* (GNSS) or *Motion Capture* (MoCap)), such as deep underground cave systems, building interiors, or extraterrestrial settings require ever higher reliance on the platform's local navigation capabilities.

This article is structured in the following way. The remainder of this section will provide a broader overview of the currently established state-of-the-art navigation solutions, followed by a selection of works more directly related to the subject of this paper. The latter sections will then describe the narrowed down algorithm configuration choices and design features used herein, together with a detailed conceptual description of the experimental setup and evaluation process. Finishing off with the

conclusion, summarizing obtained results and presenting a direction for necessary future research.

1.1. State estimation

The *Extended Kalman Filter* (EKF) is perhaps the most well-known estimation technique. Support for non-linear state transition functions is achieved by approximating the *Gaussian* distribution with a first order *Taylor Expansion* linearization. The *Unscented Kalman Filter* (UKF) instead extracts multiple weighted *sigma points* from the *Gaussian* and applies statistical linear regressions to map the density even more accurately [5]. These methods are implemented in the *robot_localization* package [6]. Other non-*Gaussian* methods, such as the *Correntropy Filter* (C-Filter) [7] use so-called *correntropy* measure comparisons between two random variables to extract higher order information from the functions. The *Adaptive Maximum Correntropy Criterion Kalman Filter* (AMCC-KF) [8] utilizes the *correntropy criterion* to improve upon the base *Gaussian* KF method. Other iterative-optimization-based methods utilizing the *factor graph* representation, which supports a wide range of tasks more flexibly, are proposed in the *fuse* package [9]. This graph formulation generally works with two node types: *variable nodes*, which represent the state estimation variables, and *factor nodes*, which convey non-linear functions tying any *variables* together with relative constraints. The most probable values of the state estimate *variables* can then be iteratively derived with respect to defined kinematic or measurement constraint *factors* [10]. Authors of [11] achieve greater pose esti-

Email address: Tomas.Horelican@vut.cz (Tomas Horelican)

mation accuracies in challenging scenarios by learning incremental pose estimates from direct *Inertial Measurement Unit* (IMU) measurements.

Kinematic proprioceptive odometry methods are an indispensable component of any localization solution and efforts have been made to also extend the solutions onto modern quadrupedal platforms [12]. Latest attempts, such as [13, 14, 15] focus on fusing multiple modalities of odometry inputs to achieve greater robustness in challenging conditions.

The most popular global pose estimation method, given a known environmental map, is *Adaptive Monte Carlo Localization* (AMCL) [16], which approximates the *pose belief* probability distribution discretely using a dynamically varying pool of weighted particles distributed throughout the map.

Recent combined global and local estimation techniques attempt to solve the *Simultaneous Localization And Mapping* (SLAM) problem using various sources of sensor data [17, 18]. The methods generally estimate a global pose trajectory of a robot using odometry and sensor measurements while gradually expanding and maintaining a consistent map representation of the environment that is also in line with all of the previous data points. Some of these projects are, however, still not ported to the new ROS2 framework. The popular *SLAM Toolbox* [19] project provides a modern package utilizing latest pose-graph optimization techniques for continuous and multi-session mapping of large dynamic interior environments using robust 2D *Light Detection And Ranging* (LiDAR) matching.

1.2. Perception

The widely used *Octomaps* [20] introduce probabilistic occupancy estimation and explicit unknown representation using hierarchical *octree* structures. The *Spatio-Temporal Voxel Layer* (STVL) [21], improves upon the traditional *voxel grid* by introducing temporal decay mechanisms instead of ray-casting clearing, which promises more authentic real-time perception of dynamic environments. Authors of *Confidence Rich Mapping* (CRM) [22] devised a new method, which incorporates a measurement confidence directly into the mapping process.

Novel approaches utilize a hierarchical fusion of multiple isolated representations coming from different information modalities (sensor types, static obstacles, inflation, scaling post-processing, traversability estimates, and other) into a final layered awareness map [23]. In a typical cost occupancy grid, values -1, 0 and 100 attribute to unknown, free and lethal pixels, respectively. A value of 99 can represent any inflation around an obstacle, and the remaining values ranging from 98 to 1 are scaled for more context-aware planning. Authors of [24] utilize the costmap semantics to incorporate a novel learning-based control together with exteroceptive vision perception into the high-level planning tasks in order to achieve more reliable state-aware navigation. Risk-aware layers utilizing the *Conditional Value-at-Risk* (CVaR) constraint have been proposed by [25, 26].

Recent breakthroughs have been made in the terrain adaptation front using *Soft Actor-Critic* (SAC) *Reinforcement Learning* (RL) [27, 28] methods for quadrupedal robots. This is done

by first encoding privileged information available in a simulation environment during training to a latent representation, with a base policy that learns to predict the next action based on a current state observation and the encoded latent. Then an adaptation module is trained to predict the latent representation only from a history of state observations without access to the privileged information [29]. This is then deployed into the real world using *sim-to-real* transfer [30] methods. The state observation can consist of just proprioceptive wheel encoder and step contact data, as was done by [31, 32]. Additionally, [33] incorporated exteroceptive visual sensor data directly into the *end-to-end* learning pipeline to achieve higher robustness in more challenging situations. Authors of [1] were able to demonstrate rapid learning capabilities without any privileged information directly in the real-world from on-board proprioceptive sensor feedback by only carefully analyzing and fine-tuning the existing approaches instead of proposing any novel solutions. A hybrid simulation/real-world approach was taken by [34], who still relied on privileged simulation data for the base policy training from proprioception, but also trained an egocentric visual module (which learns to predict the proprioceptive output) directly in the real world.

1.3. Planning

Authors of the *Regulated Pure Pursuit* (RPP) [35] algorithm introduce a latest advancement in the *pure pursuit* [36] family of local trajectory planners, which use analytical geometric solutions to find a valid control sequence guiding the robot towards a set *look-ahead* point along the path. Apart from computational improvements, this novel addition introduces new path curvature and cost-generated obstacle proximity heuristic terms to the solution, along with projecting potential future collisions. The well-known *Dynamic Window Approach* (DWA) [37, 38], which finds the most optimal control sequence from a generated set of kinodynamically constrained trajectories using a weighted objective function, is part of *Navigation2* in the form of an enhanced DWB implementation. Configurable plugin critics were implemented to influence the objective weights. A new *Model Predictive Control* (MPC) based planner *Model Predictive Path Integral* (MPPI) [39] uses relative entropy minimization within the path integral formulation. In this setting, the entire sequence of controls over a prediction horizon is optimized online and the un-executed controls are re-used in the next steps, which enables handling of non-linear dynamics by generating new behaviors. The *Navigation2* implementation performs additional stochastic perturbations on the optimized trajectory and uses a weighted critic objective function for the final selection. An advanced version of the *Timed-Elastic-Band* (TEB) [40, 41] approach performs local online *Multi-Objective* optimization of the global path, respecting execution time, obstacle distance, and kinodynamic constraints. To avoid local optima around obstacles, candidate trajectories within multiple distinctive *homology* classes are optimized in parallel and a globally optimal solution is acquired from a minimum of the objective function.

Many well-known grid-based global path planning solutions rely on some variation of the *Dijkstra* or *A** heuristic search,

which attempts to find a shortest path with minimal cost (both implemented within *Navigation2*). The *Hybrid-A** [42] algorithm improved upon the original limitations by associating a continuous vehicle state with each cell and using analytic expansions with a limited control-space kinematic model (*Dubins, Reeds-Shepp*) to guarantee a feasible path (also used by [43] at the DARPA Subterranean Challenge). The *State-Lattice* [44, 45] algorithm, on the other hand, uses an offline pre-computed set of arbitrary motion primitives to generate a *state lattice* representation used for search. This way, feasible continuous motion states for any defined chassis type are already encoded within the discretized search space. Configurable and cost-aware penalty functions and arbitrary-shaped footprint SE2 collision checking were added for both in the *Navigation2 Smac* implementations. A *lazy* version of the *Theta** planner [46], which instead of being constrained to only close neighboring vertices uses *Line-of-Sight* (LoS) checks between expanded vertex nodes to find truly shortest any-angle paths, was optimized for 3D grid search by greatly reducing the number of performed LoS checks per expansion. Additional cost-penalty functions were added for the *Navigation2* implementation. Lastly, authors of [4] utilize a risk-aware enhancement of the *D** algorithm [47, 48], in which they also take into account unknown space, risk layers, and a dynamically expandable map input, reducing the need for reactive local control.

1.4. Behaviors

Recent works, such as [49, 10] make heavy use of *Behavior Tree* (BT) structures to formalize the high level *behaviors* that guide the robotic platform through its operational tasks. The initial ideas originated within the computer game industry [50, 51] to model complex *Non-Player Character* (NPC) behaviors in a more modular and scalable fashion than what is achievable by *Finite State Machine* (FSM) models. Due to their usefulness, the robotics field (from within many others) also quickly adopted the ideas [52, 53]. As described by [54], BTs function as modular directed rooted trees, whose nodes are divided into two basic types: *control flow* and *execution*. These are then further broken into several categories, each serving a distinct functionality, for example routing internal states from *child* nodes to a *parent* node based on a defined logic, or performing a direct action and reporting the state. Complex behaviors are then modeled by chaining the individual modular components together to perform the desired task executions. Interestingly, authors of [55] first utilize *Proximal Policy Optimization* (PPO) [56] RL methods to learn low-level behaviors and then learn a high-level long-term BT representation from one human demonstration to perform complex environmental manipulation tasks. This was done with a quadruped that used only its legs for the interactions.

Authors of *Probabilistic Local and Global Reasoning on Information roadMaps* (PLGRIM) [57] merged the local, global, and high-level planning tasks into a complete solution. They encode the environmental risk awareness and exploration state representations into *Information RoadMap* (IRM) world belief structures independently for local and global contexts, and use *value learning* RL methods to train an optimal action policy

across the given belief space. The decision process is then structured in a hierarchical cascaded way going from higher-level (global) towards lower-level (local) actions. The system, however, is not universal and was developed specifically to perform full-coverage frontier exploration tasks.

2. Related Work

Recently, authors of [10] composed a comprehensive survey of most state-of-the-art algorithm implementations currently available in the full ROS2 architecture (*Navigation2* and other projects as well). They provide detailed conceptual descriptions and insights into each individual implementation. Their evaluations stem from extensive simulation experiments, however, more applied in-field reviews will still be useful.

The initial demonstration of *Navigation2* [49] as a successor to the original concept in [58] did present large-scale real-world experiments. They were, however, aimed mainly at long-term stability and, at the time, focused only on a small selection of initially implemented algorithms.

Other works like [35, 39, 40] naturally execute detailed experiments for isolated comparative evaluations that focus only on meaningful contributions to each individual algorithm. However, additional full-scale system evaluations are also essential in order to provide a more global overview.

3. Contribution

As was presented in section 2, many related works provide comprehensive evaluations only in simulated settings, or focus on single objectives (ie. isolated global planning, local planning, or state estimation) and leave out the broader behavior in a more complete combination of all tasks. This work should provide additional evaluations of most algorithms present in *Navigation2* in an identical real-world environment.

The main contributions of this article are, therefore, expected to be:

- A practical usage demonstration of nearly all advanced navigation components available within the ROS2/*Navigation2* framework at the time of writing of this article.
- A real-world experiment evaluation of current state-of-the-art navigation capabilities (particularly local and global planning algorithms) in an interior environment with some unpredictable elements.
- A unified initial configuration starting point document for the algorithm deployment on a real-world robot platform.

4. Implementation

4.1. Algorithm and equipment selection

The whole general ROS2 framework has been thoroughly described by [59]. This paper will focus more closely on the latest well-optimized and enhanced implementations of widely

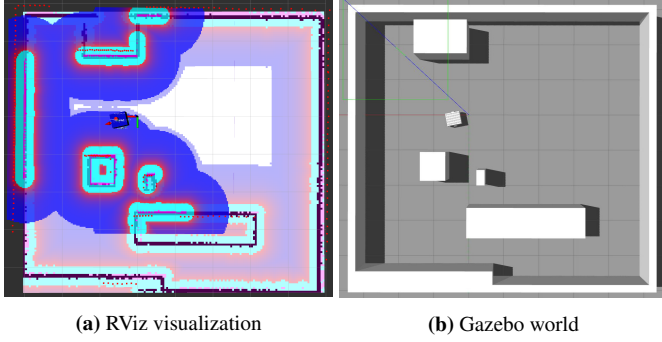


Figure 1. Simulated environment used for preliminary algorithm configuration and testing.

known algorithms within the *Navigation2* project. The main aim is to evaluate (comparatively to an extent) the different global and local planning implementations inside a consistent real-world full navigation scenario. The remaining parts of the system (such as localization or perception) and the robotic platform itself, ergo, remain constant throughout all of the experiments. Naturally, it is still essential to take these parts into account as well in any evaluation, as it may well lead to a single bottleneck (arising from insufficient localization, perception, or platform capabilities). However, by keeping the elements unchanged throughout all experiments, evaluations of the different planning and control behaviors (when faced with similar situations) can still be reasonably performed. Given the specific nature of some of the algorithms, the experimental setting of this article might not utilize their full potential most effectively¹, however it presents a consistent reproducible scenario across all of the tested combinations.

Each individual implementation was initially configured for desired behavior inside a simulated *Gazebo* environment (see Fig. 1), with subsequent fine-tuning corrections made in the real-world experimental setting.

4.1.1. Algorithms

Global path planning algorithms considered for comparisons in this paper were:

- *Navigation Function (NavFn)* using *Dijkstra* expansions,
- *Smac Hybrid-A**,
- *Smac State Lattice*,
- *Theta**.

The *Navigation2* package also provides a classic grid-search 2D A* algorithm in the *Smac 2D A** implementation and A* is an option in *NavFn* as well. The *Smac* variant promises more consistent and reliable path behavior at the cost of higher compute times. For the purposes of this comparison, *NavFn* was

¹For example the kinematically feasible path planning and arbitrary-shaped footprint SE2 collision checking of the *Smac* planners, or non-holonomic drive train support of TEB, RPP, and MPPI.

used with the *Dijkstra* expansion algorithm and the *Smac 2D A** variant was not considered, as it still shares similar conceptual behavior and would not be sensibly discernible within the more general experimental resolution for this article.

For the local trajectory controller algorithms, all four available implementations were considered:

- *Dynamic Window Approach (DWB)*,
- *Timed Elastic Bands (TEB)*,
- *Regulated Pure Pursuit (RPP)*,
- *Model Predictive Path Integral (MPPI)*.

Each of the chosen global and local planners conveys different behavioral or environmental characteristics, as was described in Section 1.3. Although most were designed and tested by the authors individually with exterior environments in mind as well, all experiments in this paper were conducted in an interior setting for easier reproducibility and base-line comparison. Furthermore, additional traversability and adaptation layers (currently only in discussion for future releases of *Navigation2*) would add more possible points of divergence between the underlying behaviors and will be more suitable for follow-up experiments.

Since some pairs of the local and global algorithms are, due to their nature, attuned to each other more appropriately than others, it would not be reasonable to test all possible combinations. As shown in Table 1, a total of eight couples of local and global algorithms were chosen based on the recommendations from [10] and the official configuration guidelines provided by the *Navigation2* project maintainers [60, 61]. The *NavFn* and *DWB* combination is the most commonly used default recommendation. The *MPPI* implementation is encouraged as the latest development in reactive motion control algorithms and is designed to be compatible with most settings. It was, therefore, tested together with all of the global planning solutions. Given the strict path-following nature of *RPP*, it was most sensible to pair it only with the feasible *Smac* planners. Finally, *Theta** and *TEB* comprise a vaguely similar alternative to the first combination, albeit conveying different intrinsic behaviors.

Perception representation was handled by the layered costmap structure, with the particular layer plugins shown in Table 2. *AMCL* was used for global pose estimation, which was fused together with wheeled-odometry data using an EKF filter provided by the *robot_localization* package. A static global map was created independently prior to experiments using the *SLAM Toolbox* package by manually operating the same robot platform in mapping mode to ensure smooth passing through any uneven areas causing large wheel slips.

4.1.2. Robot Platform

A differential drive platform (Fig. 2) with two active and two passive castor wheels was used, as this type is commonly supported by all of the tested algorithms and is a typical example of platforms used for the kinds of environments that this

Table 1. Selected global and local planner combinations for experiments.

Global	NavFn	Hybrid-A*	State Lattice	Theta*
Local	DWB	RPP	RPP	TEB
	MPPI	MPPI	MPPI	MPPI

Table 2. Layered costmap plugins used in experiments.

Global	Local
Inflation	Inflation
Obstacle	Obstacle
Static	

article is utilizing for experiments. It is equipped with a single *Velodyne HDL-32E* LiDAR (360° HFOV, 32 vertical channels), which was the only sensor used throughout all experiments. The active wheels are powered by *Maxon DCX* motors with a digital incremental encoder, controlled by *EPOS4 50/5* modules in a *ProfileVelocity* mode. A *Raspberry Pi 4B* with 4 GB of RAM communicates with the drivers through the CAN bus interface, and hosts the base chassis operation ROS2 nodes (mainly gamepad interaction interfaces, kinematics and odometry calculations, or velocity command multiplexing). An *11th-Gen Intel NUC* with an 8-core *i7-1165G7* CPU running at 2.8 GHz with *Turbo Boost* disabled and 16 GB of RAM, then hosts the more compute-intensive sensor data processing and the high-level navigation ROS2 nodes. A *MikroTik RBM33G* router-board with a 2 GHz *R11e-2HPnD* card for external wireless access serves as the main networking interface. The overall dimensions of the platform are 43.5 x 43.5 x 52 cm, including the LiDAR, wheels, and other objects extruding from the base chassis. During all experiment executions, both computers were using an *Ubuntu 22.04 GNU/Linux* distribution and ROS2 version *humble*. All core *Navigation2* algorithm packages were using versions *1.1.8*. Other hardware present on the platform was not relevant to the topic of this article.



Figure 2. The *Loki* test-bed platform used during experiments.

4.2. Configuration

4.2.1. Miscellaneous

The *SLAM Toolbox* package (used for creating the static global map) was configured with the provided default values optimized for *asynchronous online* mapping mode. The rasterizing max laser range parameter was set to 20 m, and resolution of the created map to 5 cm/pix.

The *Velodyne* was configured with default 600 rpm, and planar scan data was being extracted from the 23-rd ring in the ROS2 driver configuration. During the experiments, no additional sensor (such as an IMU) was used and the navigation solution relied only on the calculated wheeled-odometry and *Point Cloud* (or *Laser Scan*) LiDAR data.

4.2.2. Localization

AMCL localization was set to always start at an initial zero pose. Global-to-local frame transformation publishing was disabled, as EKF was responsible for the final transforms. Minimum update angle and distance parameters were set to promote high filter update rates at max velocities. During the experiments, all other parameters (including the noise estimates) were left default according to the configuration guide. Better fine-tuning of noise estimates might be preferable for any future deployments, however, even this default configuration provided satisfying results, as will be seen in Chapter 5. Resulting from the set parameters, a maximum observed pose publishing frequency was around 10 Hz. See Table B.7 for all of the relevant set parameters.

A single instance of an EKF node was used to fuse the global pose estimates from AMCL with wheeled-odometry (published at 20 Hz) velocities. Since an IMU measurement was not utilized, only a planar 2D estimation was used. A transform time offset of at least 0.1 seconds was required for the TEB controller to correctly accept the published odometry and transform messages. The *world* frame for transformations was set to be map. Handling of potentially lagged data from AMCL with filter state history was enabled. No other sources of measurements were used in the filter during experiments. The process noise covariance for the X velocity state was set to 0.1 and dynamic mode was enabled. Table B.8 shows all of the relevant configured EKF parameters.

4.2.3. Perception

Figure 3 displays the static environment map with all of the resulting costmap layers placed on top. Some of the planning and control algorithms allow for using arbitrary-shaped robot footprints, which would more accurately represent the true projected robot shape on the ground plane, and therefore enable more intelligent collision avoidance and planning. However, the most universally supported footprint shape across all of the tested algorithms was a simple circular approximation. In order to compare all algorithm behaviors within a consistent and identical setting, this was used as the primary footprint representation and any further sections of this article will always refer to the circular footprint type. Given the symmetrical square shape of the robot platform described in Section 4.1.2, this approximation is also sufficient for the practical application.

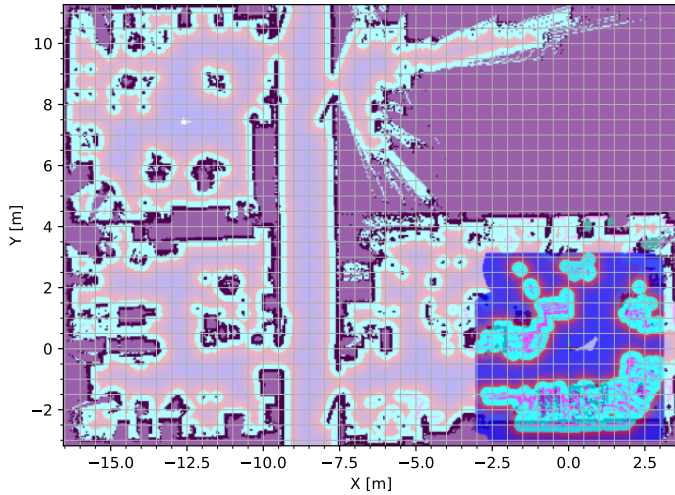


Figure 3. Map of the experiment environment overlaid with the configured costmap layers on top. In this scene, the robot was located at the map origin. Cyan sections mark an inflated lethal cost, remaining colors represent exponentially decaying cost values.

Global costmap. The global costmap, which covers the whole extent of Figure 3 was set to match the static map resolution of 5 cm/pix, and it was being updated with respect to the map frame. For the *inflation layer*, a circular footprint with a radius of 20 cm was used² to extend the lethal cost around obstacles. The remaining area up to 1.25 m was covered by exponentially decaying cost values. The *obstacle layer* was set to always mark visible obstacles within a range of 10 m, and clear visible free space within 12 m using ray-tracing. The layer was also set to include only obstacles that are within a height range of 25 cm to 60 cm. Anything below or above would not be tracked. The bottom threshold was set higher to allow more permissive planning and let the local part handle obstacles with more granularity. All other relevant global costmap parameters are also shown in Table B.9.

Local costmap. The local costmap (visualized in the bottom right of Fig. 3) was updating with a higher frequency and resolution than the global part, and relative to the robot odometry frame. A circular footprint radius of 22 cm was used² to extend lethal cost, and the inflation decay area was set within the same maximum distance as the global costmap, but with a faster decay rate. This created a steeper exponential curve, which allowed the trajectory controllers to perform local maneuvers closer to obstacles if required, while the planned global path itself always maintained its distance. The obstacle marking and clearing distance thresholds were reduced to 3 and 5 meters, respectively. The lower height threshold was decreased to 15 cm. This was sufficient for the experimental setup, however, a lower threshold, while still respecting the sensor noise floor, might be desired for practical deployment. See Table B.10 all of the relevant parameters.

²The value was calculated from the defined physical robot footprint radius reduced by the map resolution.

4.2.4. Global Path Planners

The only modified parameter for the planner server component was expected planner frequency, which was set to 10 Hz.

Each algorithm package independently allows for specifying the maximum accepted goal pose tolerance, in case a precise solution cannot be found. This was always set to be the same across all algorithms: 0.2 m.

Navigation Function. The *NavFn* plugin has little configuration options and was maintained at mostly the default values based on the implementation configuration guide, which uses the *Dijkstra* algorithm (by setting `use_astar: False`), with planning in unknown space being allowed (`allow_unknown: True`). The maximum accepted goal pose tolerance, as mentioned above, was set to 0.2 m.

Smac Hybrid-A.* This algorithm plugin was set to use the *Reeds-Shepp* motion model to also allow reversing actions. The minimum turning radius was left at the default 0.4 m to maintain the minimum curvature path behavior (even though the used differential platform would not always utilize such a path). The objective function weights were configured to mostly prefer forward motion and avoid high cost areas, to favor performing turning maneuvers only later on the path, and to allow curving paths. All other values, including final path smoothing, were left according to the documentation page default specification. See Table B.11 for all set parameters.

Smac State Lattice. This shares conceptually similar configuration with the *Hybrid-A** implementation, with differences being in the kinematic representation. The kinematic constraints are specified by the *minimum control set* configuration, which was left at the default *Ackermann* chassis vehicle type with a 0.5 m minimum turning radius. Objective weights were configured for similar behavior as the hybrid implementation (see Tab. B.12).

Theta.* The final algorithm plugin, again, provides only a small number of configuration options, namely its cost weight values, which were all left in their defaults. This algorithm does rely heavily on the costmap inflation layer and this configuration appeared to behave favorably within the configured costmap environment.

4.2.5. Local Trajectory Controllers

The general controller server component had more configuration parameters tuned. It was set to run at 10 Hz and the default *simple* goal and progress checker plugins were used. A movement of at least 10 cm in 10 seconds was set as a verification limit for successful ongoing progress towards the goal. A general tolerance, within which a goal pose should be reached was set to 0.125 m and 0.125 rad. The controllers were allowed to keep failing for a maximum of 300 ms before reporting the failure event. See Table B.13 for the full relevant parameter list.

Kinodynamic parameters, such as maximum velocities and accelerations in the linear X and angular Z (Yaw) axes, are configurable independently within each algorithm package. For the

experiments executed in this article, they were set to always be the same across all algorithms. In case of the used differential platform, the values were: ± 0.5 m/s, ± 2 rad/s, ± 2 m/s², and ± 3.2 rad/s². Some implementations also allow for specifying the goal tolerances internally. If available, they were always set to match the general 0.125 m and 0.125 rad.

Dynamic Window Approach (DWB). General (as well as trajectory generator plugin) parameters were all left at the documentation-specified defaults. The only peculiar option was linear granularity of trajectory generation, for which the default value appeared unnecessarily big (50 cm), whereas all functional examples are specifying a more reasonable value of 5 cm. The 5 cm value was also configured for experiments. The critics were tuned to avoid high obstacle cost areas, while still preferring path following in narrow corridors, and to promote straight forward motion whenever possible. See Table B.14 for the specific values.

Timed Elastic Bands (TEB). This algorithm is provided as an external package with full support in the *Navigation2* project. As such, it was configured according to the older ROS documentation page [62] and its more recent *GitHub ros2-master* branch example. Several options differed between these two sources, therefore, if a particular parameter is not present in the documentation and also not specified in Table B.15, it means the *GitHub* default was used. Since the implementation uses its own internal environmental cost representation for planning, the configuration was set to match the inflation cost settings of the local costmap. Optimization weights were tuned to demote maintaining maximum kinematic limits, and prefer low obstacle cost areas with shortest paths and execution times instead. Forward motion was preferred if possible. All the exact parameter values are shown in Table B.15.

Regulated Pure Pursuit (RPP). The RPP algorithm plugin was configured to use in-place rotation maneuvers instead of backwards motion. This setting was observed to be handling high-cost situations more reliably when executing experiments. Dynamic velocity-scaled *look-ahead* distance optimization was used to prefer high velocities in free space and track the path more precisely in constrained sections. The inflation parameters used for cost-aware obstacle-distance calculation were set to match the true costmap values. See Table B.16 for all tuned parameters, all else was left at the documentation-defined default.

Model Predictive Path Integral (MPPI). The algorithm was set to perform the optimization task for a differential motion model within a prediction horizon of 2.4 seconds, translating to 1.2 meters at max velocity (see `model_dt` and `time_steps` in Table B.17). The batch size of sampled perturbed trajectories was adjusted for the general controller frequency of 10 Hz, based on the documentation examples. 3 retry attempts were allowed before reporting a failure. Respecting path inversions in case of feasible planners was enabled. Critics were tuned to start gradually prioritizing goal over path following at 1 meter with a

hard cutoff at 0.4 meters from the goal. Path following with forward motion was preferred in obstacle-free areas, with a hand-off to goal alignment when the specified thresholds are met. The obstacle inflation parameters were, again, set to match the costmap settings.

4.2.6. Behaviors

The BT navigator server component, which manages all high-level behavior orchestration, was using mostly the default components. The global frame was left at *map*, and the robot base frame was set accordingly to the robot base link. EKF filtered odometry was used as the *odom* topic. The only notable change was the default server timeout, which was set to 200 ms. This is a remnant from older tests, where frequent timeout events were being reported with the default 20 ms. Newer validations with lower timeouts might be preferable if desired, however, it was not paramount for the experiments in this article.

The used BT structure itself was a slightly modified version of one of the provided default behaviors. It was configured to periodically re-plan the global path once every 3 seconds or immediately in case the path becomes invalid at any point in time. If planning fails, the global costmap would be cleared and one retry planning attempt would be performed before a fallback to recovery behaviors is executed. Similarly, if the local controller fails to follow the given path, it would clear the local costmap and perform one retry attempt before executing the recoveries. The recovery sequence was then configured to:

1. Clear both costmaps,
2. Wait for 5 seconds,
3. Spin 45° and Wait 1 second,
4. Spin -90° and Wait 1 second,
5. Spin 45° and BackUp -5 cm at 5 cm/s.

After 6 unsuccessful recovery attempts the BT would emit a navigation failure, and a new goal pose command would have to be issued.

4.3. Experiment Setup and Evaluation Procedure

Figure 4 shows the pre-generated SLAM map of the environment, within which all of the experiment runs were performed. The map was created using the same robot platform as was used throughout all of the experiments.

4.3.1. Main Experiment

The environment, as shown in Figure 4, still presented a challenging scenario for the navigation task, containing three sections with uneven surfaces causing potential wheel slippage, two 78 cm wide doorways leaving only 34.5 cm of empty maneuvering space when subtracting the robot size, and some highly reflective structures present in both rooms. At the time of experiment execution, it also contained new static objects not present in the a priori created map. Figure 5 displays images of the environmental setting.

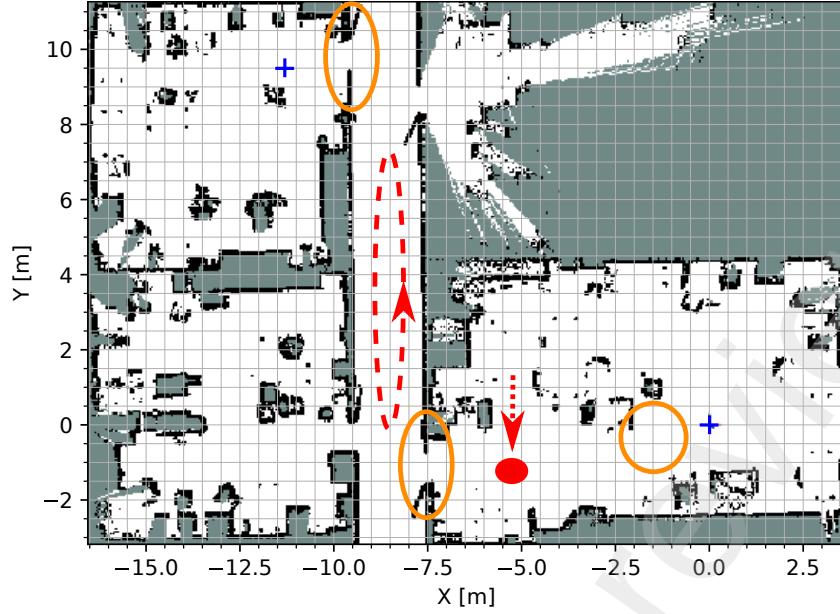


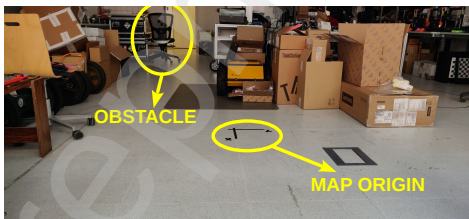
Figure 4. Map of the experiment environment. Blue crosses mark the goal poses being used during experiments. Bottom right was always the origin zero pose, where the robot was initialized, top left was always the goal pose set during (A) passes of experiments. Orange areas depict sections with uneven surfaces causing potential wheel slipping. Red paths illustrate obstacle movement during dynamic experiments. Bottom room ((A) only): obstacle was moved into the robot path and left at the marked position. Hall ((A) and (B)): A person was passing the robot always using the right side, forcing a dynamic reaction, first approaching from behind and then from the front.



(a) Top room



(b) Hall



(c) Bottom room

Figure 5. Experiment environment setting.

The robot was always initialized at the map origin location (bottom right in Fig. 4) at the start of every experiment execution. For each algorithm combination, an experiment execution consisted of two separate passes:

- (A) Navigating from the initial position (bottom in Fig. 4) to the second room (top in Fig. 4): -11.3 m (X), 9.5 m (Y), 0 rad (Yaw).
- (B) Navigating from the second room back to the initial position: 0 m (X), 0 m (Y), 0 rad (Yaw).

Each complete experiment was executed three times and this was done with two separate situational scenarios:

- a static environment with no dynamic obstacles,
- a dynamic environment with obstacle encounters along the way.

In total, 48 complete experiment executions were performed amounting to more than 2 km of potential total traveled distance, excluding any failures or undesired path diversions.

During the experiments, some of the combinations reached unrecoverable localization failures at some point during their execution. In each case, only the first fully complete successful re-run of, both, pass (A) and (B) was kept for processing. If pass (A) was completed successfully, but the solution failed irrecoverably during pass (B), the whole experiment execution was discarded and re-run again from the beginning. These *total* failures happened almost exclusively during dynamic experiments and their main cause was excessive wheel slipping from

Table 3. Total failures during dynamic obstacle experiment execution attempts.

	NavFn		Hybrid-A*	State Lattice	Theta*	
Total failures	DWB	MPPI	RPP	RPP	TEB	MPPI
Pass (A)	1/1 ¹	2	2	-	2	-
Pass (B)	-	3	4	1	-	1

¹ This was the only failure that happened during a static experiment attempt.

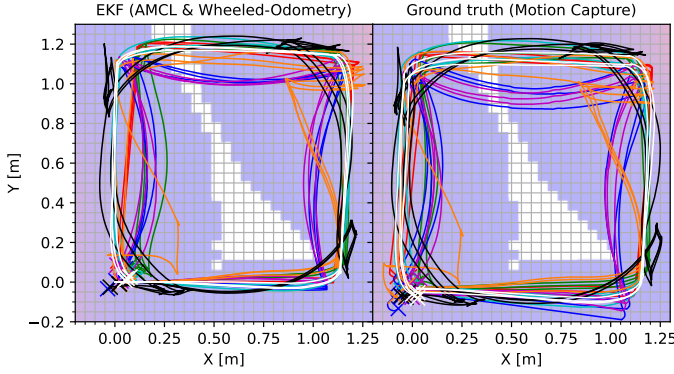


Figure 6. Localization system validation experiment with all algorithm combinations. X symbols mark the final poses. Underlying background shows the costmap. Data colors correspond to Figure 8.

the uneven surface sections. They will not be included in the final data evaluation, as they would not be generally repeatable. However, it is essential to mention them here for fidelity. Table 3 shows all of the encountered failure counts.

An area surrounding the initial position (right half of the bottom room) was covered by a MoCap system, which had its origin frame synchronized with the map origin and was, therefore, providing precise global position data valid in the map frame. This was not used for localization in any part of the experiment, and instead served as a ground truth position and orientation for evaluating the *true* final robot pose when returning to the initial position in pass (B).

An external computer was used to send goal pose commands to the robot. These were being continuously sent every 5 seconds until the goal was reached and no further movement was observed after several commands. This was set to maintain continued operation even when navigation failed completely, but not cause frequent interventions to successful progress.

In addition, both experiment passes ((A) and (B)) were also executed once by manually controlling the robot from start to a goal pose. This provided an ideal human operator reference behavior baseline for comparison.

The NUC wall-clock time (recorded at 20 Hz with nanosecond precision) served as a reference for duration measurements. All data was being recorded by the standard ROS2 *bag* file structure interface.

4.3.2. Localization Validation Experiment

A short set of experiments was also performed to validate the isolated localization (and perception) behavior in an ideal simple scenario. These were performed under full coverage of

the MoCap system with no uneven surface areas along the path. All of the components were already configured as described in Section 4.2 and here, the MoCap output was used as ground truth for all robot poses.

The robot was initialized in the starting zero position at the map origin. It then received four consecutive goal pose commands, covering a square path with a 1.2 m arm length, which ended back at the origin position. The goal commands were being sent on by one, each next goal was sent after the previous one was successfully completed as reported by the BT navigator. The exact goal values were:

1. 1.2 m (X), 0.0 m (Y), 1.57 rad (Yaw),
2. 1.2 m (X), 1.2 m (Y), 3.14 rad (Yaw),
3. 0.0 m (X), 1.2 m (Y), -1.57 rad (Yaw),
4. 0.0 m (X), 0.0 m (Y), 0.0 rad (Yaw).

This was always repeated three times for all selected algorithm combinations. This way, an expected behavior could be validated when being used with all of the planning algorithm combinations. Figure 6 shows all of the resulting followed trajectories according to the filtered EKF odometry and ground truth MoCap.

4.3.3. Evaluation

All recorded data processing took place on a machine with a *Ubuntu 20.04* distribution and ROS2 version *foxy*. To ensure correct compatibility with the newer system, an LXD container environment with *Ubuntu 22.04* and ROS2 *humble* was utilized for processing.

Results from all three complete experiment executions were always processed using Equation 1 to calculate a final *mean* (μ) and *standard deviation* (σ) estimate for each result category.

$$\mu = \frac{1}{3} \sum_{i=1}^3 A_i, \quad (1)$$

$$\sigma = \sqrt{\frac{1}{3} \sum_{i=1}^3 |A_i - \mu|^2},$$

Understandably, it is not sensible to discuss any meaningful statistical analysis results when such a small sample size is used. However, given the practical feasibility limitations of executing a larger amount of experiment repetitions, this sample size should be sufficient to provide a general picture overview and distinguish any occurring nondeterministic outlier behaviors resulting from the unknown environmental elements. To account for this, the result data will also be analyzed with more redundant representation methods, which should convey a more complete view of the whole situation.

Total Time (T_{total}). The start time of each individual experiment pass ((A) or (B)) was measured as time of the first received goal pose message. The end time was always taken as time of the last EKF odometry message, where the absolute value of linear

X or angular Z (Yaw) velocity component was above a certain threshold (see Eq. 2).

$$t_{last_odom} = \begin{cases} t_{odom}, & \text{if } |v_x| > 0.06 \vee |\omega_z| > 0.08 \\ t_{last_odom}, & \text{otherwise} \end{cases} \quad (2)$$

Duration measurements were then estimated as a time difference between the last odometry message above thresholds and the first received goal pose message. This was calculated individually for each of the three experiment executions and the final *mean* and *standard deviation* values were taken from these results by substituting:

$$A_i = T_{total,i} = t_{last_odom,i} - t_{first_goal,i}$$

in Equation 1. Since neither of these messages had valid time stamp information included implicitly during experiments, the closest recorded clock time was always taken for the calculation.

True Distance to Goal ($d_{goal,i}^{true}$, $\alpha_{goal,i}^{true}$). Given that the MoCap frame origin was aligned with the map frame origin, raw position and orientation values measured by the MoCap system were directly used to calculate the *true* linear and angular distances from the goal pose of pass (B). This was, again, calculated individually for each execution and by substituting:

$$A_i = d_{goal,i}^{true} = \sqrt{x_{mocap,i}^2 + y_{mocap,i}^2} \text{ and} \\ A_i = \alpha_{goal,i}^{true} = \alpha_{Z,i}^{mocap}$$

in Equation 1, the *mean* and *standard deviation* values were taken from these three results.

Odometry Estimate Distance to Goal ($d_{goal,i}^{odom}$, $\alpha_{goal,i}^{odom}$). For reference, the same steps were also taken to calculate linear and angular distances from goal using the EKF filtered odometry position and orientation values. This could be done for both, the (A) and (B) passes, while taking into account the goal pose coordinate values as well, extending the Equation 1 substitutions to:

$$A_i = d_{goal,i}^{odom} = \sqrt{(x_{odom,i} - x_{goal,i})^2 + (y_{odom,i} - y_{goal,i})^2} \text{ and} \\ A_i = \alpha_{goal,i}^{odom} = \alpha_{Z,i}^{odom}.$$

Failure Counts ($N_{failures}^{controller}$, $N_{failures}^{planner}$, $N_{failures}^{BT}$). Log outputs from the controller, planner, and BT navigator servers were used to count failure events. The system would report corresponding failures in case of the following situations:

- The local controller would fail if no valid local trajectory could be found or no progress was being made towards the goal within the specified thresholds.
- The planner would fail if no valid path could be planned towards the goal within the specified tolerance.
- Each of these failures would cause the BT to execute specified recovery actions. The BT would then emit a failure after 6 failed recovery attempts, and a new goal pose would have to be issued to re-start the navigation process again.

The corresponding counts from each execution were then used in Equation 1 for the final results:

$$A_i = N_{failures,i}^{controller}, \\ A_i = N_{failures,i}^{planner}, \text{ and} \\ A_i = N_{failures,i}^{BT}.$$

Average Velocities (v_x^{avg} , ω_z^{avg}). Linear and angular velocities throughout each experiment execution duration were extracted from the fused EKF odometry data. The final *mean* and *standard deviation* results from all three experiment executions were calculated using averages of the absolute values by substituting:

$$A_i = v_{X,i}^{avg} = \frac{1}{N} \sum_{n=1}^N |v_{X,i}^n| \text{ and} \\ A_i = \omega_{Z,i}^{avg} = \frac{1}{N} \sum_{n=1}^N |\omega_{Z,i}^n|,$$

where $N \Leftrightarrow$ number of values within experiment duration

in Equation 1.

4.3.4. Localization Validation Evaluations

The same final *true* and *odom* estimate distances from goal as described in Section 4.3.3 were also calculated from the validation experiments data.

Since these experiments were always executed under full MoCap coverage, additional evaluations could be performed on the complete traveled trajectories shown in Figure 6. Experiment start and end time values were also estimated using the same procedure as described in Section 4.3.3 and were used to extract pose data that is valid within the actual experiment execution time boundaries. The MoCap pose data (published at 100 Hz) could then be filtered out by taking each fifth value to match the odometry data (published at 20 Hz). With the MoCap poses serving as ground truth trajectory data, *root mean squared error* measures were calculated for each individual axis (linear X, Y, and angular Z (Yaw)) using Equation 3. The trajectories from all three experiment executions for each algorithm were concatenated into one continuous data set, and that was used to calculate the total error estimates.

$$RMS E_{AXIS}^{traj} = \sqrt{\frac{1}{N+M+K} \sum_{n=1}^{N+M+K} (A_n - \hat{A}_n)^2}, \quad (3)$$

where $A_n \in \{x_{mocap,n}, y_{mocap,n}, \alpha_{Z,n}^{mocap}\}$,

$\hat{A}_n \in \{x_{odom,n}, y_{odom,n}, \alpha_{Z,n}^{odom}\}$,

$N \Leftrightarrow$ number of values in first execution,

$M \Leftrightarrow$ number of values in second execution, and

$K \Leftrightarrow$ number of values in third execution.

Table 4. Averaged final pose distances to zero according to MoCap and EKF filtered odometry from three validation experiment executions, and total full trajectory RMS errors, for all algorithm combinations.

	NavFn				Hybrid-A*				State Lattice				Theta*			
	DWB		MPPI		RPP		MPPI		RPP		MPPI		TEB		MPPI	
	μ	σ	μ	σ	μ	σ	μ	σ	μ	σ	μ	σ	μ	σ	μ	σ
d_{goal}^{true} [mm]	51	22	28	6	94	45	41	20	51	19	68	34	75	15	68	12
α_{goal}^{true} [mrad]	133	10	72	26	345	5	98	13	347	48	155	3	65	66	124	21
d_{goal}^{odom} [mm]	113	4	121	0	48	32	84	12	66	25	60	23	27	6	65	12
α_{goal}^{odom} [mrad]	111	2	65	19	345	17	74	10	326	56	97	6	89	73	85	8
$RMS E_X^{traj}$ [mm]	64		71		56		51		40		56		30		37	
$RMS E_Y^{traj}$ [mm]	53		59		81		68		81		53		36		64	
$RMS E_{Yaw}^{traj}$ [mrad]	21		15		68		38		38		32		31		28	

5. Results

5.0.1. Evaluation Experiments

Table 4 shows results from the validation experiments. Under these ideal conditions, the complete navigation solution was able to reach the desired final goal pose to within around 9 cm and 9° (0.155 rad) using any algorithm combination. The *true* linear distances are all well within the configured goal tolerance of 12.5 cm. The *true* angular values extend beyond the configured 7° (0.125 rad) in two cases: *NavFn*+*DWB* and *State Lattice*+*MPPI*. When looking at the values reported by odometry it is clear that in all cases, the algorithmic solution assumed the tolerances were met, and these deviations can be attributed minor localization discrepancies. The only evident outliers are the RPP combinations, which were consistently reaching as high as 20° (0.347 rad) angular distances from zero, as reported mutually by MoCap as well as odometry. This is explained by two factors. The configured maximum rotational velocity was causing large overshoots when the goal orientation was reached and, since the goal command was always sent only once, no additional corrections were being made after the *goal reached* event was reported. These angular distances will be substantially lower in some of the main experiment executions given the fact that in those experiments, the solution always received follow-up goal commands even after the goal was reached initially, giving it more opportunities to correct its final pose. Relatively high angular data variances between executions are seen in the *Theta**+*TEB* combination, which can be attributed to the highly dynamic nature of *TEB*. Deviations of up to 9 cm and 3° (0.058 rad) between Mocap and odometry can be seen in the final pose data. The total trajectory RMS deviations are all under 7 cm, 8 cm, and 4° (0.068 rad), for the X, Y, and Yaw axis respectively. Given these results, the expected precision of the localization component under ideal conditions should be reaching the desired tolerances requested from the full navigation solution, and any further deviations should attribute to unexpected situation reactions and different algorithm behaviors.

5.1. Main Experiments

5.1.1. Static Environment

The most problematic section to deal with was the uneven surface floor close to the initial position (bottom right in Fig.

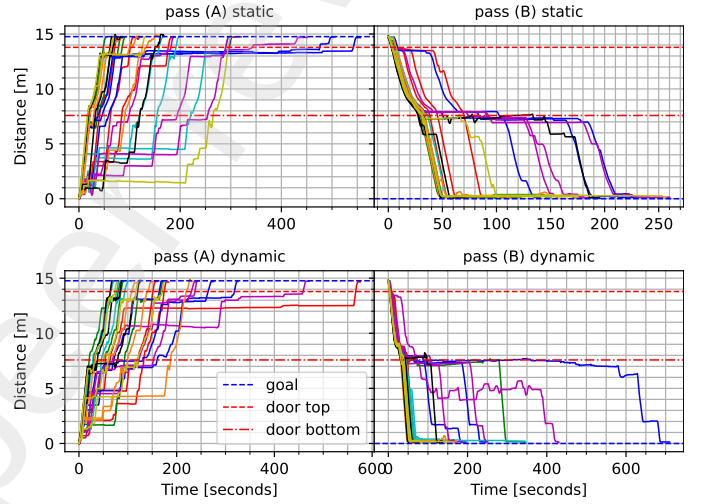


Figure 7. Linear distance to map origin from EKF filtered odometry with respect to execution time for all algorithm combinations during experiments. Data colors correspond to Figure 8, white is replaced by yellow here.

4). Since this environmental feature was not being explicitly accounted for in the perception component, all of the tested solutions navigated through this section with a degree of localization loss. In all cases, the solutions relied on the same recovery actions to correct the pose estimate and it would then be dependent on how each algorithm combination was able to continue performing the navigation task. Figure 8 shows the traveled odometry trajectories plotted on top of a representative map layer and separated by the experiment characteristic (static or dynamic) and the execution pass ((A) or (B)). It can be seen that almost all of the trajectories convey an amount of diversion from a naturally expected ideal path during the first half of pass (A). However, all managed to converge before exiting the room. Some path diversions can also be seen after any uneven doorway section is crossed and also near the end of pass (B). Table 5 presents the acquired averaged results from static environment experiment executions. All of the solutions were able to successfully accomplish the full navigation tasks with only a single hard failure described in Section 4.3.1. Apart from this,

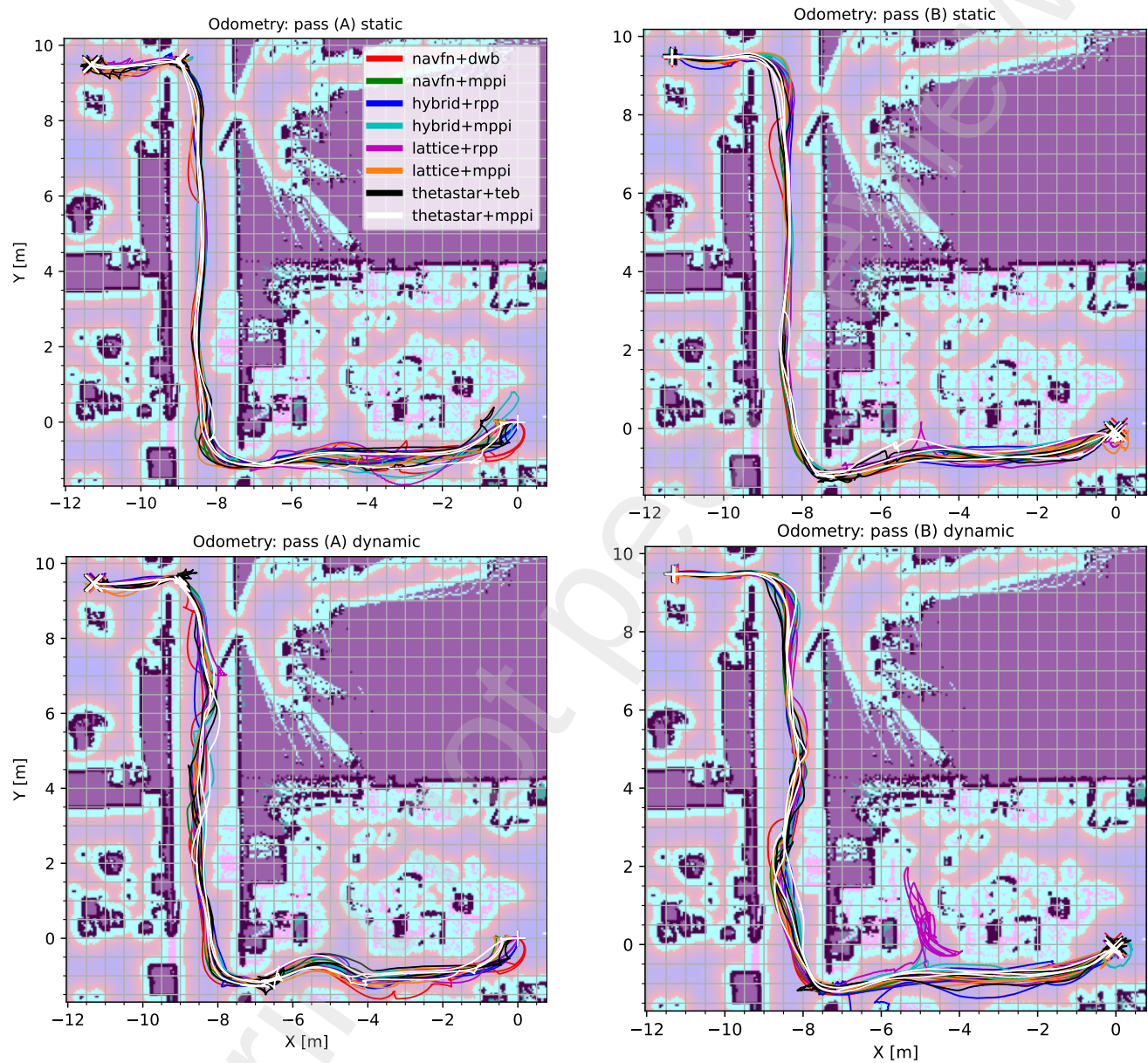


Figure 8. EKF filtered odometry trajectories from all algorithm combinations during experiments. The + symbol always marks the starting pose, and X always marks the goal pose. The underlying costmap image was taken from a single time point at the map origin and does not necessarily represent the exact costmap layout throughout all of the experiment executions.

Table 5. Averaged results from three experiment executions for all tested algorithm combinations (static environment). Best results in each category (if relevant) are shown in bold. Manual execution is shown for reference.

Manual			NavFn		Hybrid-A*				State Lattice				Theta*					
			DWB		MPPI		RPP		MPPI		RPP		MPPI		TEB		MPPI	
			μ	σ	μ	σ	μ	σ	μ	σ	μ	σ	μ	σ	μ	σ	μ	σ
T_{total} [minutes]	(A)	1.16	2.17	0.72	1.31	0.27	7.59	1.8 ¹	3.71	0.94	5.34	1.77	2.03	0.49	1.77	0.73	2.82	1.63
	(B)	1.07	1.54	0.35	2.06	0.33	3.40	0.75	1.40	0.78	3.48	0.61	1.77	0.72	1.76	1.07	2.97	1.05
d_{goal}^{true} [mm]	(A)	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
	(B)	32	89	45	190	24	356	118	623	325	497	246	390	197	349	211	391	226
α_{goal}^{true} [mrad]	(A)	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
	(B)	14	84	18	116	74	43	19	391	184	263	354	206	137	235	180	241	250
d_{goal}^{odom} [mm]	(A)	16	98	10	10	2	76	19	51	30	72	10	65	32	66	12	32	6
	(B)	22	117	2	72	37	120	9	101	29	127	16	122	27	89	40	108	23
α_{goal}^{odom} [mrad]	(A)	15	113	4	101	4	31	41	96	11	45	22	90	6	45	35	87	9
	(B)	28	106	4	63	44	53	31	59	35	41	2	68	47	67	13	68	30
$N_{failures}^{controller}$ [-]	(A)	-	16	8	7	4	195	43	17	5	74	28	11	5	12	6	15	8
	(B)	-	4	4	8	2	56	19	4	4	68	12	5	2	11	11	17	3
$N_{failures}^{planner}$ [-]	(A)	-	37	16	1	0	47	30	37	11	61	27	10	10	31	34	62	83
	(B)	-	2	1	1	1	32	14	0	0	16	8	0	0	31	37	12	16
$N_{failures}^{BT}$ [-]	(A)	-	2	1	0	0	16	4	3	1	8	3	1	1	2	2	4	6
	(B)	-	0	0	0	0	5	2	0	0	5	0	0	0	2	3	1	1
v_X^{avg} [$\frac{mm}{s}$]	(A)	322	195	62	319	52	54	14	131	21	83	26	225	45	273	84	203	99
	(B)	341	247	54	186	29	114	27	334	129	112	18	260	116	295	122	151	59
ω_Z^{avg} [$\frac{mrad}{s}$]	(A)	170	206	19	205	21	164	6	121	12	175	43	187	16	308	17	166	14
	(B)	108	213	39	103	19	190	27	121	22	177	29	124	34	246	43	105	68

¹ First experiment execution. Due to excessive wheel slippage, the solution reached a deadlock state at the second door opening encounter in its path (top in Fig. 4), and after approx. 6.9 minutes of total execution time, the robot was operated manually through the doorway. Autonomous mode was resumed 20 cm after the doorway. Manual operation time of around 5 seconds was not subtracted from the result.

during one of the executions using *Hybrid-A*+RPP* the robot remained stuck in a wheel-slipping position and was unable to cross the second room doorway in pass (A) after several autonomous recovery attempts. This situation was, however, recoverable by manually moving the robot through the door section and resuming autonomous operation afterwards (see Tab. 5, Note 1).

Within this static environment, the shortest execution time of around 3.4 minutes (when both passes are combined) was achieved using the *NavFn+MPPI* combination. These were also the most consistent, as is evident by the run-to-run deviations. The *Theta*+TEB* and *NavFn+DWB* followed closely (3.5 minutes and 3.7 minutes, respectively) with also slightly larger deviations between executions. The close results reflect a higher degree of similarity between each local and global planner variant. It is also evident that pairing the MPPI local controller with any global planner generally improved duration results when compared to the alternative. The only digression was *Theta*+MPPI*, which has a considerably worse combined execution time of 5.8 minutes. Figure 7 shows an odometry estimated linear distance from the zero origin point plotted with respect to experiment execution time. It is visible that one of the executions using *Theta*+MPPI* was losing a considerable amount of time in the beginning section of pass (A). If this single outlier is omitted, the total time would decrease only down to 4.7 minutes. A large amount of time was also repeatedly being spent in pass (B) at the goal location, where the controller appeared to be stuck in a local minimum when attempting to perform final goal alignment maneuvers. This is also evident

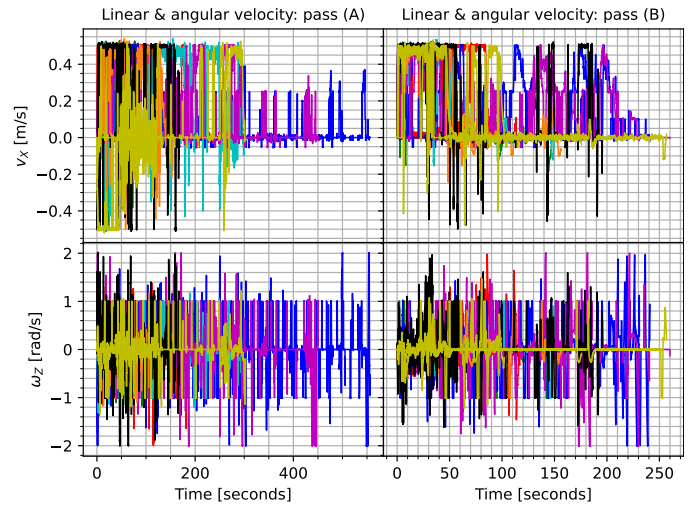


Figure 9. Linear and angular velocity profiles from EKF filtered odometry for all algorithm combinations during static environment experiments. Colors correspond to Figure 8, white is replaced by yellow here.

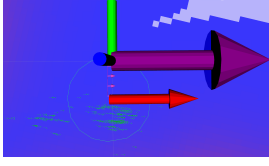


Figure 10. Local minimum position for the MPPI controller. Smaller red arrow: EKF filtered odometry pose. Larger purple arrow: desired goal pose.

from the plotted velocities in Figure 9 for pass (B). This particular final alignment behavioral glitch was observed across all used MPPI combinations if the robot managed to find it self at the pose shown in Figure 10. Therefore, all of the results using MPPI suffer a degree of uniformly worsened time duration results. At the time of writing, the source of the problem is not clear, since this does not appear to be related to crossing low-to-high cost boundaries, path following, path-to-goal critic hand-off behaviors, or specific path shape issues.

The longest execution times are seen from *Hybrid-A*+RPP* and *State Lattice+RPP* with 11 minutes and 8.8 minutes, respectively. These combinations also experienced highest run-to-run deviations. As seen in Table 5, the RPP combinations reported the highest amount of component failures and most often relied on recovery actions. These combinations most often suffered from localization losses at the doorway boundaries (leading to wheel-slipping states) and, as is visible in Figure 7, spent most time recovering from these situations. This would most definitely be a bottleneck of the localization and perception sections, however, it is still notable that other algorithm combinations were able to recover from these situations more promptly. A small amount of recovery behavior executions was also observed when passing around the high-cost corner areas with highly curved paths. This would be most likely attributed to short-cutting behaviors leading the robot closer to the high cost area before managing to drive around the corner fully, and is expected to be further remedied by lowering the *look-ahead* distances.

Given the unaccounted for floor irregularity near the goal location, *true* distance measures from goal did increase up to 62 cm and 22° (0.391 rad) (Tab. 5) (or 36 cm and 8° (0.136 rad) if outlying situations are excluded) in the static experiments. After passing this section during most experiment executions, the algorithms would gradually keep correcting the pose estimate and navigating closer towards the goal until reaching a final best position. In some cases, however, the system would reach the specified goal according to its odometry estimate before any impactful pose corrections could be made, resulting in a higher *true* distance measure. Figures A.14 and A.15 present a more complete picture, where all of the experiment results for all executions are visualized in violin-plot formats. For this low sample case, they show all three executions in a compact representation using the *min*, *max*, and *median* values, and also visualize an estimated distribution density trace emerging from the data points. As seen from A.15, the *NavFn+DWB*,

NavFn+MPPI, and *Hybrid-A*+RPP* completed all executions without any outliers. Outlier states happened twice for *Hybrid-A*+MPPI* and only once in all other combinations. Even when all the outlier cases are omitted, *NavFn+DWB* still recovered reaching the smallest distances of 9 cm and 5° (0.084 rad) with a relatively high repeatability and still maintained leading execution times. The following *NavFn+MPPI* combination reached 19 cm and 7° (0.116 rad) on average. The *State Lattice+RPP* and *Hybrid-A*+RPP* combinations would reach the best single angular distance results of 1° (0.013 rad) and 2° (0.043 rad), respectively, as was alluded to in the beginning of this chapter. However, at the same time they would reach the largest linear distance results with 33 cm and 36 cm, respectively. In practice, a final rotate-in-place maneuver would be advisable to correct the estimated pose and give the solution an additional nudge to re-navigate if any dis-localization is still not rectified. Again, a general trend of improved results with MPPI is noticeable.

When looking at the *odometry* estimated distance results in Table 5, it can be seen that some of the linear distance values do reach beyond the specified tolerance of 12.5 cm, particularly at least once in all four combinations with *Hybrid-A** and *State Lattice*, and the *Theta*+MPPI* combination. This is visible only within the (B) passes and in all these cases, navigation did not re-initiate even after several additional goal pose commands. Upon further inspection, all three global planners were always placing their final path point slightly shifted from zero to -2.5 cm (X) and -2.5 cm (Y). The largest odometry deviation from tolerance amounted to 3 cm, which indicates that this final path point shift made the largest contribution to the final reached position error.

5.1.2. Dynamic Obstacle Environment

The final set of experiments was performed with additional dynamic obstacle encounters (as was shown in Figure 4) along the same traveled path. Nearly all failure cases mentioned in Section 4.3.1 emerged during these dynamic experiments. However, when these specific instances are disregarded, all of the algorithm combinations were able to perform the full navigation tasks while also successfully reacting to the obstacle encounters. Trajectory changes as well as more diversity between the paths are clearly visible in the bottom half of Figure 8. It is also important to mention that two experiment executions had to be manually recovered. The *State Lattice+RPP* combination reached a planning deadlock state during one execution after it passed through the second doorway in pass (B) with excessive localization loss due to wheel slipping. This situation is clearly visible in the bottom right section of Figure 8, where the solution was navigating back and forth between two locations and was unable to recover a correct localization estimate. This state was recoverable manually by operating the robot towards the intended path and rotating in place, as is described in Table 6, Note 1. The second instance happened during one execution with *Hybrid-A*+RPP*, where the robot remained stuck inside the second door opening of pass (B). Again, this was recovered by manually moving the robot through the entrance and rotating in place, after which, autonomous operation was resumed.

Table 6. Averaged results from three experiment executions for all tested algorithm combinations (dynamic obstacle encounters). Best results in each category (if relevant) are shown in bold. Manual execution is shown for reference.

Manual			NavFn		Hybrid-A*				State Lattice				Theta*					
			DWB		MPPI		RPP		MPPI		RPP		MPPI		TEB		MPPI	
			μ	σ	μ	σ	μ	σ	μ	σ	μ	σ	μ	σ	μ	σ	μ	σ
T_{total} [minutes]	(A)	1.18	5.11	3.17	1.98	0.66	4.31	1.04	1.45	0.20	5.32	1.79	2.81	0.80	1.56	0.40	2.14	0.66
	(B)	1.04	1.11	0.07	2.99	2.03	6.40	3.86 ²	2.87	2.08	4.82	1.73 ¹	2.11	0.82	1.46	0.59	1.69	0.76
d_{goal}^{true} [mm]	(A)	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
	(B)	44	143	121	140	87	278	31	488	216	726	301	360	263	127	103	348	195
α_{goal}^{true} [mrad]	(A)	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
	(B)	20	105	68	100	43	53	10	321	256	523	346	311	404	90	51	326	333
d_{goal}^{odom} [mm]	(A)	24	109	8	12	3	68	27	32	4	54	36	73	30	53	10	36	5
	(B)	74	77	26	74	35	129	2	107	24	132	16	128	20	76	37	124	35
α_{goal}^{odom} [mrad]	(A)	67	112	3	95	3	38	45	97	13	42	35	89	14	43	30	87	4
	(B)	42	110	5	83	31	34	17	67	46	47	12	74	40	46	25	103	10
$N_{controller\ failures}$ [-]	(A)	-	50	20	15	4	116	38	11	1	102	17	17	3	12	3	21	7
	(B)	-	3	1	11	6	122	82	14	9	80	30	9	3	5	1	8	2
$N_{planner\ failures}$ [-]	(A)	-	45	29	28	17	17	10	5	3	64	45	37	18	12	11	31	26
	(B)	-	1	1	45	62	76	87	4	2	46	14	0	0	17	20	2	1
$N_{failures}^{BT}$ [-]	(A)	-	6	1	2	1	8	3	0	0	11	4	3	1	1	1	2	2
	(B)	-	0	0	3	5	13	11	0	0	8	3	0	0	1	1	0	0
v_X^{avg} [$\frac{mm}{s}$]	(A)	330	103	44	220	51	94	25	278	34	79	19	158	42	313	60	219	64
	(B)	369	332	27	213	133	80	32	223	105	104	14	219	105	330	101	262	99
ω_Z^{avg} [$\frac{mrad}{s}$]	(A)	260	164	24	230	30	186	24	234	19	190	27	169	33	415	42	218	31
	(B)	187	163	46	138	45	163	16	154	58	210	32	121	45	336	67	142	32

¹ First experiment execution. The solution reached a deadlock state after excessive wheel slippage when passing through the second door opening in its path (bottom in Fig. 4), and after approx. 5.9 minutes of total execution time, the robot was manually operated approx. 1.3 meters back towards the expected path, rotated 180° counter-clockwise, and 180° clockwise. After this, autonomous mode was resumed. Manual operation time of around 32 seconds was not subtracted from the result.

² Third experiment execution. Due to excessive wheel slippage, the solution reached a deadlock state at the second door opening encounter in its path (bottom in Fig. 4), and after approx. 9.3 minutes of total execution time, the robot was manually operated approx. 1.3 meters forward through the doorway, rotated 360° counter-clockwise, and 360° clockwise. After this, autonomous mode was resumed. Manual operation time of around 43 seconds was not subtracted from the result.

Due to a disjointed orientation of the local and global frames when manual control was acquired, a substantial path diversion out of the navigable area is visible in the bottom right section of Figure 8. However, once the manual recovery operations were completed, the solution managed to continue and fully complete its task in autonomous mode (see Tab. 6, Note 2).

The shortest and also most consistent combined execution times in this experiment scenario were achieved by *Theta**+TEB with around 3 minutes. It is likely attributed to the highly dynamic behavior of TEB being best suited to quickly react to the unexpected obstacle situations. This was followed by *Theta**+MPPI and *Hybrid-A**+MPPI with 3.8 minutes and 4.3 minutes, respectively. The same trend of improvement with MPPI as in the static experiments is also noticeable here. When comparing the TEB and MPPI options, Figures 7 and A.16, do show a relatively consistent preference for the TEB algorithm in pass (A) with no apparent outliers. However, the longer time periods for MPPI spent near the goal in pass (B) are visible here in Figures 7 and 11 as well. A noticeable outlier is clearly visible in Figures 11 and A.16 for *NavFn*+DWB in pass (A). Figure 7 shows that the algorithm combination spent a large amount of time waiting before entering the second room. The controller appeared to reach a local minimum state at the position shown in Figure 12. The source of this problem is not known at the time of writing of this paper, however, it is expected to be related to the tuning of the obstacle critics relative to the path critics. If this outlier case is omitted, however, the combination

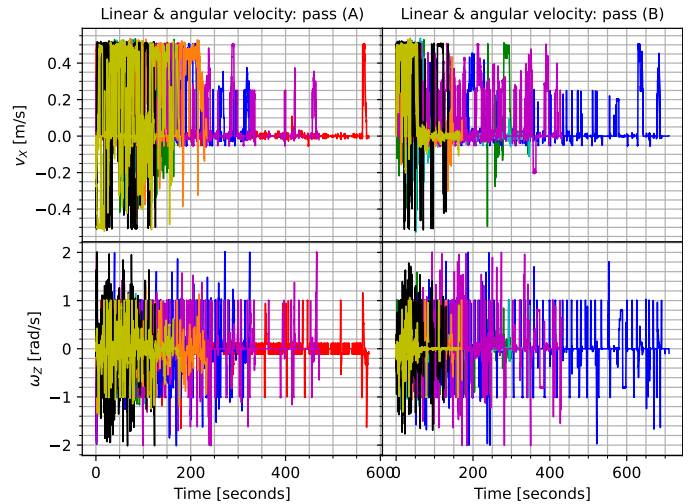


Figure 11. Linear and angular velocity profiles from EKF filtered odometry for all algorithm combinations during dynamic obstacle experiments. Colors correspond to Figure 8, white is replaced by yellow here.

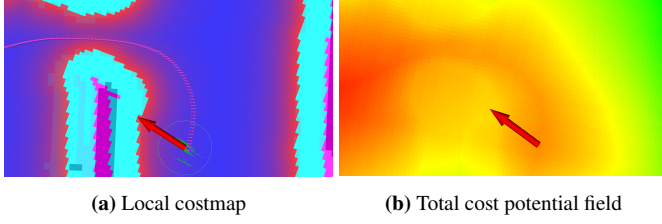


Figure 12. Local minimum position for the DWB controller, (b) was captured after experiments at the approximate location of the event. Red arrow: EKF filtered odometry pose. Purple line: Global path.

would reach an average of 4 minutes of total execution time, bringing it on par with the similar TEB and MPPI algorithms.

Both RPP combinations again reached worst average execution times here, which were also the least consistent between repetitions with around 10.7 minutes for *Hybrid-A*+RPP* and 10.1 minutes for *State Lattice+RPP*. When looking at Figure A.17, the manual recovery executions did result in larger outlier deviations from the mean, especially for *Hybrid-A*+RPP*. Even if these outlier cases are disregarded, the durations would decrease only down to around 8 minutes for *Hybrid-A*+RPP* and 9 minutes for *State Lattice+RPP*. The pattern of higher reliance on recovery behaviors for RPP is also still visible from the high failure rates and lower average velocities in Table 6.

The dynamic experiments were completed with final *true* distance to goal measures reaching up to 73 cm and 30° (0.523 rad), or 34 cm and 8° (0.14 rad) if outlier cases are disregarded. Similar counts of outlier goal pose states as in static experiments were also observed within the dynamic scenario (see Fig. A.17). The *NavFn+DWB*, *NavFn+MPPI*, *Hybrid-A*+RPP*, and *Theta*+TEB* completed all executions without any outlier results. Two situations happened with *State Lattice+RPP* and all the remaining combinations had one each. Even when the outliers are excluded, the smallest distance to goal results were achieved using *Theta*+TEB* with 13 cm and 5° (0.09 rad). This would be followed by *NavFn+MPPI* with 14 cm and 6° (0.1 rad). Again, when the outlier cases are omitted, both RPP combinations would be reaching low angular distances of 2° (0.043 rad) and 3° (0.053 rad) for *State Lattice+RPP* and *Hybrid-A*+RPP*, respectively. The smallest angular result would, however, be reached using *State Lattice+MPPI* with 1° (0.026 rad). The largest linear distance results in these experiments were from *Hybrid-A*+MPPI* and *State Lattice+RPP* with 34 cm and 30 cm, respectively. Table 6 also shows that according to the *odometry* distance estimates the same protrusions beyond the specified linear distance to goal tolerance are visible. This is a result of the same behavior as with the static experiments.

5.1.3. Discussion

Even with the simplest possible perception and localization configuration, the available planning algorithms were able to perform all navigation tasks and mostly recover from any failure states. All of the experiment executions (static and dynamic) were completed without any collision events even when

substantial localization losses were occurring. This also validates the overall system reliability of all combinations in complex situations. When comparing the static and dynamic scenario results, no obvious differences are noticeable. All combinations reacted adequately to the changing situation (by either promptly re-planning or stopping) without major issues. The increased total failure counts during dynamic experiments as mentioned in Chapter 4.3.1 are notable, however, these should more closely relate to acquiring better solutions in the perception, localization and adaptation areas. A minor increase in execution times and failure rates for some combinations is expected when tasked with reacting to the obstacle situations. However, some combinations occasionally showed even better results in the dynamic experiments, which should indicate that creating the additional obstacle encounters did not cause a greater variance than the unaccounted for ground discontinuities.

As was seen, the more liberal controller implementations generally fared better across all experiments in the presented metrics. Their reactive behaviors proved to help in resolving the difficult situations more rapidly before resorting to recovery actions. Especially notable is the ability to enable backwards motion to back out from constrained situations using TEB, MPPI, and RPP (although this option was not used in RPP for the experiments). However, if predictable path following and strictly safe maneuvers (like scaling velocities with cost or path curvature) are required, the RPP algorithm with thoughtful parameter optimization and a feasible global path planner should still be a preferable option. When comparing the MPPI algorithm as an alternative to DWB, it did generally show more natural and stable behavior, and a more user-friendly parameter configuration API. The presented experiment results are within tolerance between each other (given the low sample size), but a slight preference for MPPI is still recognizable. The biggest setbacks for MPPI were the local minimum states shown in Figure 10 and if this is remedied, it is expected that the results would safely approach the manual baseline, as even now the best overall results were achieved using MPPI. TEB did show faster reactions to difficult situations with its more dynamic behaviors and generally approached highest velocities throughout the experiments, however, configuring the controller for exact desired and controlled behavior was much less intuitive.

Given the unified experiment conditions, advanced SE2 collision checking or kinematically constrained path feasibility guarantees of *Hybrid-A** and *State Lattice* were not utilized fully. Follow-up experiments with non-holonomic platforms and more direct utilization of the applicable features are still desirable. When deployed in the full navigation scenario, all planning algorithms were able to reactively generate valid paths given the changing environmental state (obstacle events or localization shifts). When comparing the MPPI combinations with all global planning algorithms, the results generally lie within the same margins. Stemming from this, more valuable conclusions can be made based on the algorithm utility and features rather than the presented results. If kinematic constraints are not an issue, the non-feasible types might be a strong consideration due to their ability to find a wider range of valid paths, with the *NavFn* variant still achieving best results. How-

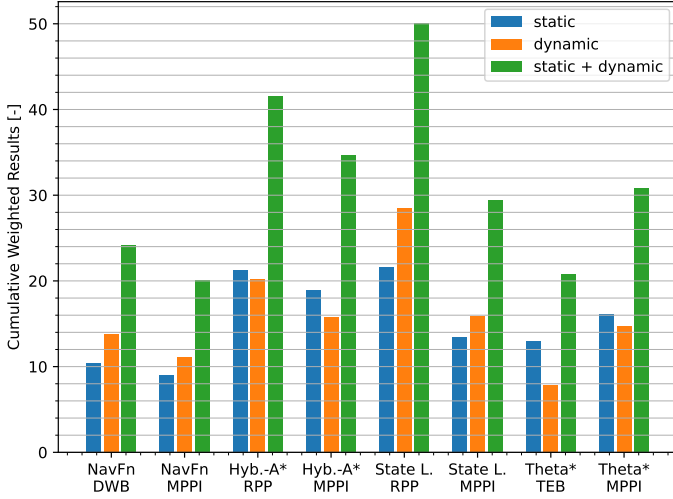


Figure 13. Cumulative sum of weighted result values for each algorithm combination and experiment.

ever, if such limits must be obeyed, the *State Lattice* implementation might be preferable particularly for its ability to support a wider range of abstract drivetrain types.

In order to somewhat simplify all of the acquired experiment results under one unified comparative view, a simple weighted cumulative objective function, summarizing all results is presented in Equation 4.

$$\frac{1}{3} \sum_{result} w_{result} \sum_{i=1}^3 |result|, \quad (4)$$

$$\begin{aligned} \text{where } w_{\text{duration}} &= 1, \\ w_{\text{distance}} &= 10, \\ w_{\text{angle}} &= 10, \text{ and} \\ w_{\text{failures}} &= 0.01. \end{aligned}$$

The reasoning behind this is that all of the shown result values should in an ideal situation limit be reaching zero. The only exception are the time duration values, which should nonetheless still be as low as possible within physically reasonable constraints. Average velocity results were not considered for this evaluation as a single preference is not as easily predicated. Highest (or closest to maximum) velocities might be favorable to complete the objectives as fast as possible, however, lower averages can be preferable when dealing with many high-cost situations. Moreover, this result is already reflected in the time duration. An average of the absolute value of all three experiment executions was always taken as the input *result* to Equation 4. Then, the averaged standard SI unit numerical values themselves can be abstracted as unitless scores, which were weighted to bring all result types onto a similar magnitude order with equal contributions. When all of the results for each algorithm combination are accumulated this way, it is expected that lower numbers should generally reflect better comparative results. Figure 13 shows this boiled-down cumulative comparison.

Shortcomings. As was mentioned, the experimental environment did on one hand present a challenging but still reasonable setting that stress tested the various algorithm behaviors. On the other hand, the perception and localization component bottlenecks did in some cases create edge case results, which must be examined when making any conclusions. A higher sample of experimental executions would certainly also help in acquiring more definite conclusions.

In order to maintain the main focus of this article directed towards the planning components within the full navigation task context, rather simpler perception and localization configurations were used. However it is evident that more robust solutions in these particular areas could in fact also help in reducing the higher variances between experiments.

The presented evaluations were dependent on the most universal set of features that are applicable to all of the algorithms in the same way. Experiments with direct utilization of the more specialized capabilities should still be performed.

6. Conclusion

This paper presented a global overview and demonstration of the planning algorithms available in *Navigation2* in the context of a whole system navigation task. As was shown in Chapter 5, all tested solutions were able to successfully perform the given tasks without any collision events even when subjected to unfavorable conditions. Therefore, comparative evaluations can be made as long as they are understood within the experimental conditions of this article, and the shortcomings are taken into account. These algorithms were being tested under identical conditions and therefore a minimum common functionality set was used across all the algorithm implementations. Additional experiments, utilizing also the non-holonomic drivetrain and arbitrary-shaped vehicle footprint support capabilities, should add even more insightful conclusions.

The biggest complication still lies in the need for careful understanding and configuration of each algorithm for the given task, even though the provided default recommendations already greatly simplify the work. As was shown, all of the solutions can be tuned to achieve desired reliable behavior to a reasonable extent. And naturally, the currently obtained results can still improve with even more careful tuning.

As a side note, this paper also emphasizes that a more robust multi-modal localization solution should be used to increase reliability when extreme conditions are encountered. Further work, incorporating better perception and adaptation techniques outlined in Chapter 1 should be a promising natural continuation of the system in order to increase the navigation robustness under challenging conditions and start pushing the applicability into even more unstructured exterior environments. Additional planning and behavioral validations could then be made within the extended context.

Appendix A. Extended experiment results

Appendix B. Relevant or modified module parameters

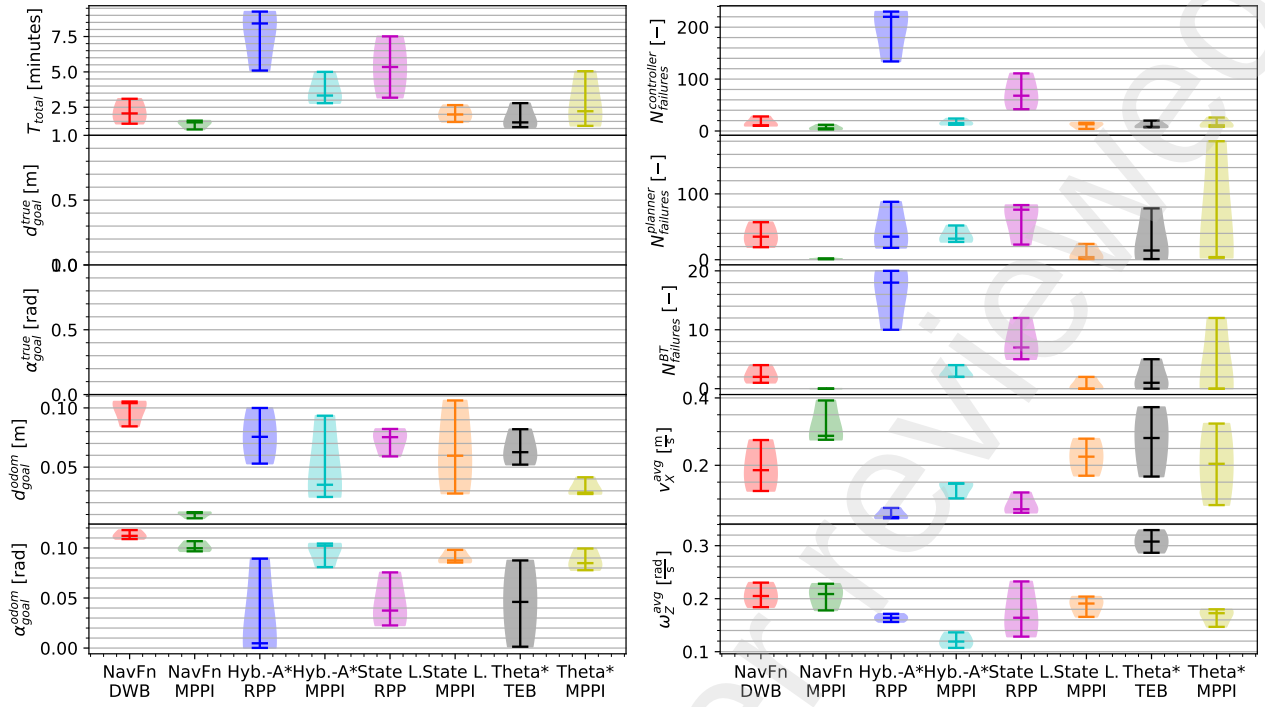


Figure A.14. Full results from three experiment executions for all tested algorithm combinations (static environment, pass (A)). Markers show the minimum, maximum, and median data point, areas visualize an estimated density distribution from the data. Colors also correspond to Figure 8, white is replaced by yellow here.

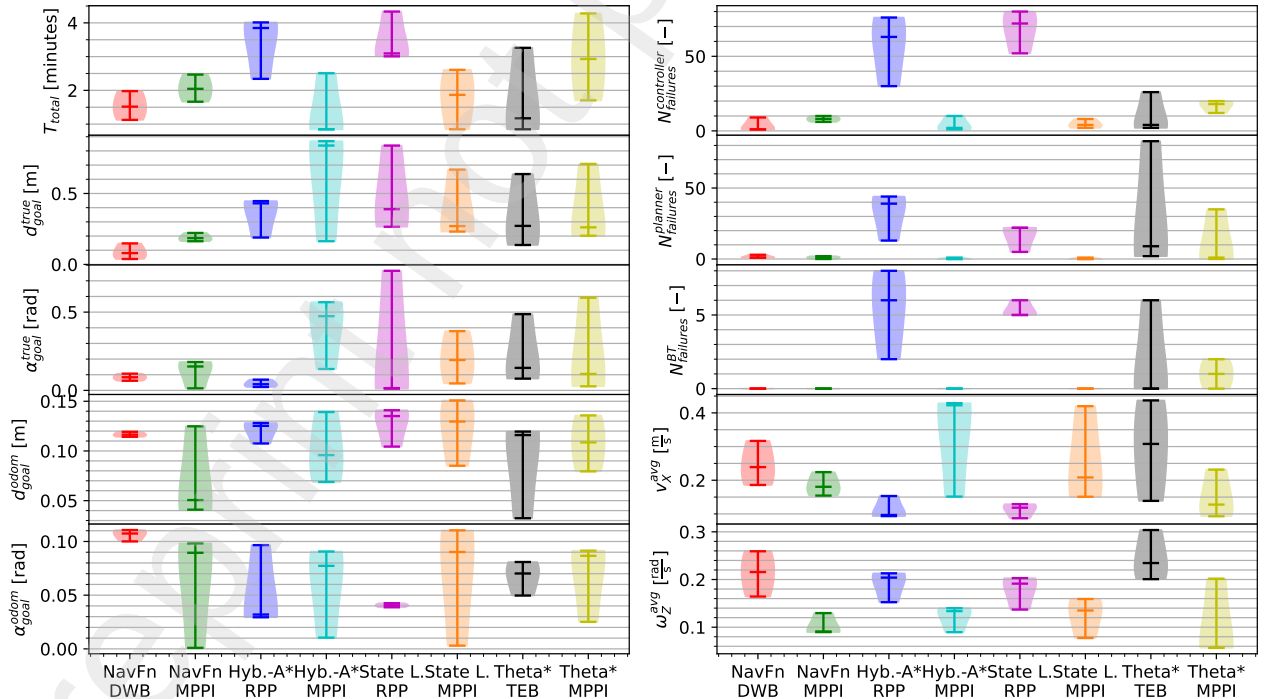


Figure A.15. Full results from three experiment executions for all tested algorithm combinations (static environment, pass (B)). Markers show the minimum, maximum, and median data point, areas visualize an estimated density distribution from the data. Colors also correspond to Figure 8, white is replaced by yellow here.

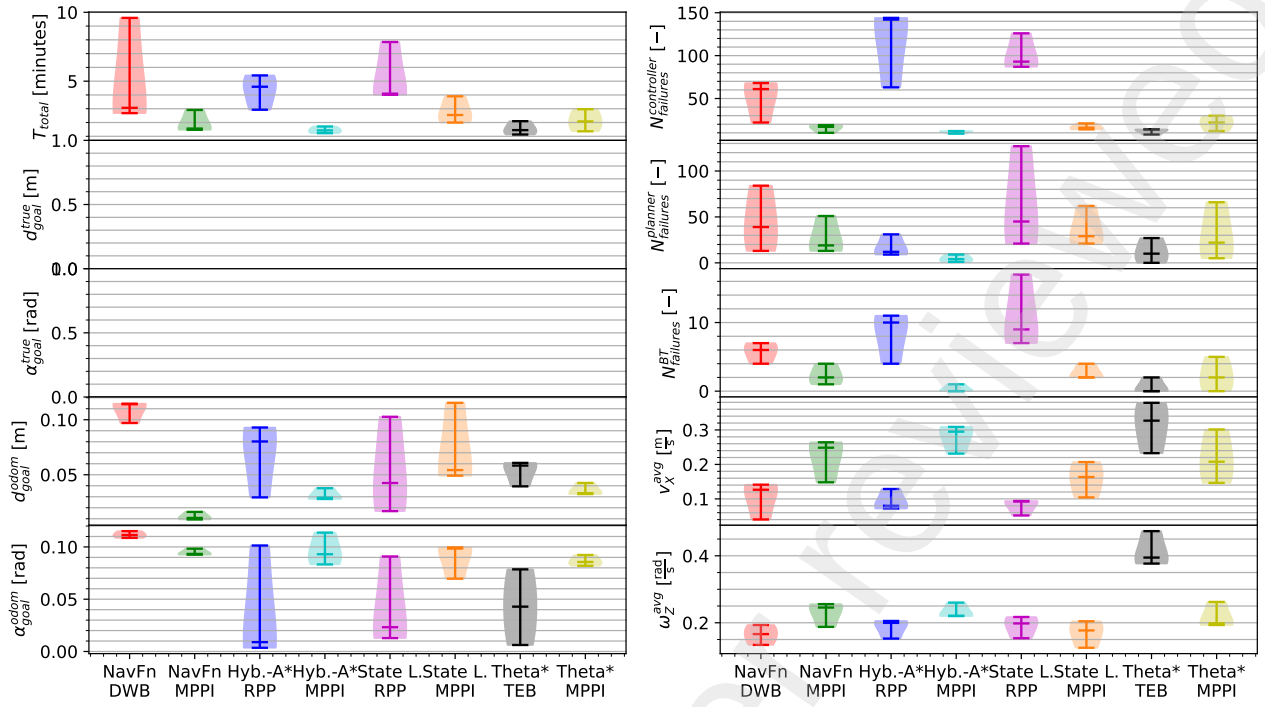


Figure A.16. Full results from three experiment executions for all tested algorithm combinations (dynamic obstacle encounters, pass (A)). Markers show the minimum, maximum, and median data point, areas visualize an estimated density distribution from the data. Colors also correspond to Figure 8, white is replaced by yellow here.

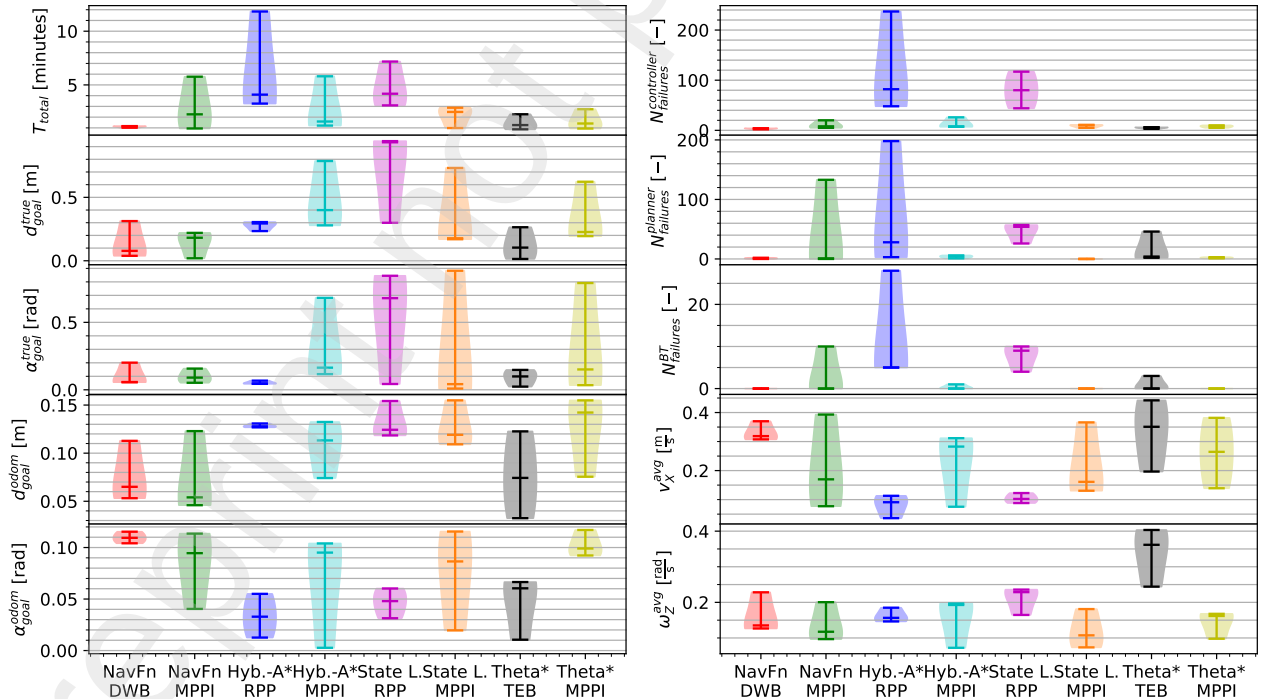


Figure A.17. Full results from three experiment executions for all tested algorithm combinations (dynamic obstacle encounters, pass (B)). Markers show the minimum, maximum, and median data point, areas visualize an estimated density distribution from the data. Colors also correspond to Figure 8, white is replaced by yellow here.

Table B.7. AMCL

```

initial_pose: {0.0, 0.0, 0.0, 0.0}
laser_max_range: 50 m
tf_broadcast: False
base_frame_id: robot footprint
global_frame_id: map
odom_frame_id: robot odom
update_min_a: 0.02 rad
update_min_d: 0.03 m
laser_model_type: likelihood field
robot_model_type: differential

```

Table B.8. EKF

```

frequency: 20 Hz
two_d_mode: True
transform_time_offset: 0.1 s
map_frame: map
odom_frame: robot odom
base_link_frame: robot footprint
world_frame: map
smooth_lagged_data: True
history_length: 0.1 s
odom_velocities: X, Y, Yaw
AMCL_poses: X, Y, Yaw
<data>_relative: False
<data>_differential: False
<data>_nodelay: False
dynamic_process_noise_covariance: True

```

Table B.9. Global costmap

```

update_frequency: 1 Hz
publish_frequency: 1 Hz
global_frame: map
robot_base_frame: robot base link
track_unknown_space: True
resolution: 0.05 m/pix
robot_radius: 0.2 m
always_send_full_costmap: True
<static>_subscribe_to_updates: True
<static>_transform_tolerance: 0.1 s
<obstacle>_max_obstacle_height: 0.6 m
<obstacle>_min_obstacle_height: 0.25 m
<obstacle>_expected_update_rate: 0.5 s
<obstacle>_obstacle_max_range: 10.0 m
<obstacle>_obstacle_min_range: 0.25 m
<obstacle>_raytrace_max_range: 12.0 m
<obstacle>_raytrace_min_range: 0.0 m
<obstacle>_clearing: True
<obstacle>_marking: True
<inflation>_cost_scaling_factor: 3.0
<inflation>_inflation_radius: 1.25 m
<inflation>_inflate_around_unknown: True

```

Table B.10. Local costmap

```

update_frequency: 5 Hz
publish_frequency: 2 Hz
global_frame: robot odom
robot_base_frame: robot base link
rolling_window: True
width: 6 m
height: 6 m
resolution: 0.03 m/pix
robot_radius: 0.22 m
always_send_full_costmap: True
<obstacle>_max_obstacle_height: 0.6 m
<obstacle>_min_obstacle_height: 0.15 m
<obstacle>_expected_update_rate: 0.5 s
<obstacle>_obstacle_max_range: 3.0 m
<obstacle>_obstacle_min_range: 0.25 m
<obstacle>_raytrace_max_range: 5.0 m
<obstacle>_raytrace_min_range: 0.0 m
<obstacle>_clearing: True
<obstacle>_marking: True
<inflation>_cost_scaling_factor: 6.0
<inflation>_inflation_radius: 1.25 m

```

Table B.11. Smac Hybrid-A*

```

allow_unknown: True
tolerance: 0.2 m
max_planning_time: 3 s
motion_model_for_search: Reeds-Shepp
reverse_penalty: 2.0
change_penalty: 0.4
non_straight_penalty: 1.05
cost_penalty: 3.5
retrospective_penalty: 0.5

```

Table B.12. Smac State Lattice

```

allow_unknown: True
allow_reverse_expansion: True
tolerance: 0.2 m
max_planning_time: 3.0 s
reverse_penalty: 2.0
change_penalty: 0.4
non_straight_penalty: 1.05
cost_penalty: 3.5
rotation_penalty: 5.0
retrospective_penalty: 0.5

```

Table B.13. general controller

controller_frequency: 10.0 Hz
 failure_tolerance: 0.3 s
 <progress_checker>.required_movement_radius: 0.1 m
 <progress_checker>.movement_time_allowance: 10.0 s
 <goal_checker>.xy_goal_tolerance: 0.125 m
 <goal_checker>.yaw_goal_tolerance: 0.125 rad

Table B.14. DWB

linear_granularity: 0.05 m
 GoalAlign.scale: 24.0
 GoalDist.scale: 24.0
 RotateToGoal.scale: 32.0
 RotateToGoal.slowness_factor: 2.0
 PathAlign.scale: 32.0
 PathDist.scale: 64.0
 BaseObstacle.scale: 0.08
 ObstacleFootprint.scale: 0.04
 PreferForward.scale: 5.0
 PreferForward.theta_scale: 10.0
 PreferForward.penalty: 1.0
 Oscillation.scale: 3.0
 Twirling.scale: 2.0

Table B.15. TEB

max_global_plan_lookahead_dist: 1.0 m
 feasibility_check_no_poses: 3
 global_plan_overwrite_orientation: True
 allow_init_with_backwards_motion: True
 footprint_model.type: *circular*
 footprint_model.radius: 0.22 m
 min_turning_radius: 0.0 m
 xy_goal_tolerance: 0.125 m
 yaw_goal_tolerance: 0.125 rad
 min_obstacle_dist: 0.25 m
 inflation_dist: 1.25 m
 obstacle_poses_affected: 15
 dynamic_obstacle_inflation_dist: 1.25 m
 costmap_converter_plugin: *DBSMCCH polygons*
 costmap_converter_rate: 5 Hz
 obstacle_cost_exponent: 6.0
 weight_max_vel_x: 0.5
 weight_max_vel_theta: 1.0
 weight_acc_lim_x: 0.05
 weight_acc_lim_theta: 0.1
 weight_kinematics_forward_drive: 5.0
 weight_optimal_time: 10.0
 weight_shortest_path: 3.0
 weight_obstacle: 50.0
 weight_inflation: 0.06
 weight_dynamic_obstacle: 50.0
 weight_dynamic_obstacle_inflation: 0.03
 selection_cost_hysteresis: 5.0
 selection_obst_cost_scale: 1.0
 roadmap_graph_area_width: 5.0 m
 h_signature_prescaler: 0.5
 obstacle_heading_threshold: 0.45
 selection_alternative_time_cost: True

Table B.16. RPP

min_lookahead_dist: 0.5 m
 max_lookahead_dist: 1.0 m
 lookahead_time: 5.0 s
 min_approach_linear_velocity: 0.1 m/s
 approach_velocity_scaling_dist: 0.75 m
 max_allowed_time_to_collision_up_to_carrot: 1.0 s
 curvature_lookahead_dist: 1.0 m
 cost_scaling_dist: 0.7 m
 cost_scaling_gain: 1.0
 inflation_cost_scaling_factor: 6.0
 use_velocity_scaled_lookahead_dist: True
 use_rotate_to_heading: True

Table B.17. MPPI

<code>motion_model:</code>	<code>diff drive</code>
<code>batch_size:</code>	<code>8000</code>
<code>time_steps:</code>	<code>24</code>
<code>model_dt:</code>	<code>0.1 s</code>
<code>retry_attempt_limit:</code>	<code>3</code>
<code>prune_distance:</code>	<code>1.0 m</code>
<code>enforce_path_inversion:</code>	<code>True</code>
<code>ConstraintCritic.cost_weight:</code>	<code>4.0</code>
<code>GoalAngleCritic.cost_weight:</code>	<code>3.0</code>
<code>GoalAngleCritic.threshold_to_consider:</code>	<code>0.4 m</code>
<code>GoalCritic.cost_weight:</code>	<code>5.0</code>
<code>GoalCritic.threshold_to_consider:</code>	<code>1.0 m</code>
<code>PathAngleCritic.cost_weight:</code>	<code>2.0</code>
<code>PathAngleCritic.offset_from_furthest:</code>	<code>4</code>
<code>PathAngleCritic.max_angle_to_furthest:</code>	<code>1.0 rad</code>
<code>PathAngleCritic.threshold_to_consider:</code>	<code>0.4 m</code>
<code>PathAlignCritic.cost_weight:</code>	<code>14.0</code>
<code>PathAlignCritic.max_path_occupancy_ratio:</code>	<code>0.05 %/100</code>
<code>PathAlignCritic.trajectory_point_step:</code>	<code>4</code>
<code>PathAlignCritic.threshold_to_consider:</code>	<code>0.4 m</code>
<code>PathAlignCritic.use_path_orientations:</code>	<code>True</code>
<code>PathFollowCritic.cost_weight:</code>	<code>8.0</code>
<code>PathFollowCritic.offset_from_furthest:</code>	<code>5</code>
<code>PathFollowCritic.threshold_to_consider:</code>	<code>0.6 m</code>
<code>ObstaclesCritic.cost_scaling_factor:</code>	<code>6.0</code>
<code>ObstaclesCritic.inflation_radius:</code>	<code>0.25 m</code>
<code>ObstaclesCritic.consider_footprint:</code>	<code>True</code>
<code>PreferForwardCritic.cost_weight:</code>	<code>5.0</code>
<code>PreferForwardCritic.threshold_to_consider:</code>	<code>0.4 m</code>
<code>TwirlingCritic.cost_weight:</code>	<code>10.0</code>

Acknowledgments

The work has been performed in the project AI4CSM: Automotive Intelligence for/at Connected Shared Mobility No 101007326/8A21013. The work was co-funded by grants of Ministry of Education, Youth and Sports of the Czech Republic and Chips Joint Undertaking (Chips JU). The work was supported by the infrastructure of RICAIP that has received funding from the European Union's Horizon 2020 research and innovation programme under grant agreement No 857306 and from Ministry of Education, Youth and Sports under OP RDE grant agreement No CZ.02.1.01/0.0/0.0/17_043/0010085. The completion of this paper was made possible by the grant No. FEKT-S-23-8451 - "Research on advanced methods and technologies in cybernetics, robotics, artificial intelligence, automation and measurement" financially supported by the Internal science fund of Brno University of Technology.

References

- [1] I. Kostrikov, L. M. Smith, S. Levine, Demonstrating A Walk in the Park: Learning to Walk in 20 Minutes With Model-Free Reinforcement Learning, in: *Proceedings of Robotics: Science and Systems*, Daegu, Republic of Korea, 2023. doi:10.15607/RSS.2023.XIX.056.
- [2] H. A. Lauterbach, C. B. Koch, R. Hess, D. Eck, K. Schilling, A. Nüchter, The eins3d project — instantaneous uav-based 3d mapping for search and rescue applications, in: *2019 IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR)*, 2019, pp. 1–6. doi:10.1109/SSRR.2019.8848972.
- [3] G. Nikolakopoulos, A. Agha, Pushing the limits of autonomy for enabling the next generation of space robotics exploration missions, *Computer* 54 (2021) 100–103. doi:10.1109/MC.2021.3055914.
- [4] B. Lindqvist, S. Karlsson, A. Koval, I. Tevetzidis, J. Haluška, C. Kanellakis, A. akbar Agha-mohammadi, G. Nikolakopoulos, Multimodality robotic systems: Integrated combined legged-aerial mobility for subterranean search-and-rescue, *Robotics and Autonomous Systems* 154 (2022). URL: <https://www.sciencedirect.com/science/article/pii/S0921889022000756>. doi:<https://doi.org/10.1016/j.robot.2022.104134>.
- [5] S. Thrun, W. Burgard, D. Fox, *Probabilistic Robotics*, Intelligent Robotics and Autonomous Agents series, MIT Press, 2006. URL: https://books.google.cz/books?id=k_y0QgAACAAJ.
- [6] T. Moore, D. Stouch, A generalized extended kalman filter implementation for the robot operating system, in: *Proceedings of the 13th International Conference on Intelligent Autonomous Systems (IAS-13)*, Springer, 2014.
- [7] G. T. Cinar, J. C. Príncipe, Hidden state estimation using the correntropy filter with fixed point update and adaptive kernel size, in: *The 2012 International Joint Conference on Neural Networks (IJCNN)*, 2012, pp. 1–6. doi:10.1109/IJCNN.2012.6252730.
- [8] S. Fakoorian, A. Santamaria-Navarro, B. T. Lopez, D. Simon, A.-a. Agha-mohammadi, Towards robust state estimation by boosting the maximum correntropy criterion kalman filter with adaptive behaviors, *IEEE Robotics and Automation Letters* 6 (2021) 5469–5476. doi:10.1109/LRA.2021.3073646.
- [9] S. Williams, fuse, 2023. URL: <https://github.com/locusrobotics/fuse>, accessed: 2023-10-26.
- [10] S. Macenski, T. Moore, D. V. Lu, A. Merzlyakov, M. Ferguson, From the desks of ros maintainers: A survey of modern & capable mobile robotics algorithms in the robot operating system 2, *Robotics and Autonomous Systems* 168 (2023). URL: <https://www.sciencedirect.com/science/article/pii/S092188902300132X>. doi:<https://doi.org/10.1016/j.robot.2023.104493>.
- [11] R. Buchanan, M. Camurri, F. Dellaert, M. Fallon, Learning inertial odometry for dynamic legged robot state estimation, in: A. Faust, D. Hsu, G. Neumann (Eds.), *Proceedings of the 5th Conference on Robot Learning*, volume 164 of *Proceedings of Machine Learning Research*, PMLR, 2022, pp. 1575–1584. URL: <https://proceedings.mlr.press/v164/buchanan22a.html>.
- [12] R. Hartley, J. Mangelson, L. Gan, M. Ghaffari Jadidi, J. M. Walls, R. M. Eustice, J. W. Grizzle, Legged robot state-estimation through combined forward kinematic and preintegrated contact factors, in: *2018 IEEE International Conference on Robotics and Automation (ICRA)*, 2018, pp. 4422–4429. doi:10.1109/ICRA.2018.8460748.
- [13] R. Thakker, N. Alatur, D. D. Fan, J. Tordesillas, M. Paton, K. Otsu, O. Toupet, A.-a. Agha-mohammadi, Autonomous off-road navigation over extreme terrains with perceptually-challenging conditions, in: B. Siciliano, C. Laschi, O. Khatib (Eds.), *Experimental Robotics*, Springer International Publishing, Cham, 2021, pp. 161–173.
- [14] A. Agha, K. Otsu, B. Morrell, D. D. Fan, R. Thakker, A. Santamaria-Navarro, S.-K. Kim, A. Bouman, X. Lei, J. Edlund, M. F. Ginting, K. Ebadi, M. Anderson, T. Pailevanian, E. Terry, M. Wolf, A. Tagliabue, T. S. Vaquero, M. Palieri, S. Tepsuporn, Y. Chang, A. Kalantari, F. Chavez, B. Lopez, N. Funabiki, G. Miles, T. Touma, A. Buscicchio, J. Tordesillas, N. Alatur, J. Nash, W. Walsh, S. Jung, H. Lee, C. Kanellakis, J. Mayo, S. Harper, M. Kaufmann, A. Dixit, G. Correa, C. Lee, J. Gao, G. Merewether, J. Maldonado-Contreras, G. Salhotra, M. S. D. Silva, B. Ramtoul, Y. Kubo, S. Fakoorian, A. Hatteland, T. Kim, T. Bartlett, A. Stephens, L. Kim, C. Bergh, E. Heiden, T. Lew, A. Cauligi, T. Heywood, A. Kramer, H. A. Leopold, C. Choi, S. Daftry, O. Toupet, I. Wee, A. Thakur, M. Feras, G. Beltrame, G. Nikolakopoulos, D. Shim, L. Carlone, J. Burdick, Nebula: Quest for robotic autonomy in challenging environments; team costar at the darpa subterranean challenge, 2021. arXiv:2103.11470.
- [15] M. Tranzatto, F. Mascarich, L. Bernreiter, C. Godinho, M. Camurri, S. Khattak, T. Dang, V. Reijgwart, J. Loeje, D. Wisth, S. Zimmermann, H. Nguyen, M. Fehr, L. Solanka, R. Buchanan, M. Bjelonic, N. Khedekar, M. Valceschini, F. Jenelten, M. Dharmadhikari, T. Homberger, P. D. Petris, L. Wellhausen, M. Kulkarni, T. Miki, S. Hirsch, M. Montenegro, C. Papachristos, F. Tresoldi, J. Carius, G. Valsecchi, J. Lee, K. Meyer, X. Wu, J. Nieto, A. Smith, M. Hutter, R. Siegwart, M. Mueller, M. Fallon, K. Alexis, Cerberus: Autonomous legged and aerial robotic exploration in the tunnel and urban circuits of the darpa subterranean challenge, 2022. arXiv:2201.07067.
- [16] D. Fox, Kld-sampling: Adaptive particle filters and mobile robot localization, *Advances in Neural Information Processing Systems (NIPS)* 14 (2001) 26–32.
- [17] K. Ebadi, M. Palieri, S. Wood, C. Padgett, A.-a. Agha-mohammadi, Dare-slam: Degeneracy-aware and resilient loop closing in perceptually-degraded environments, *Journal of Intelligent and Robotic Systems* 102 (2021). URL: <https://doi.org/10.1007/s10846-021-01362-w>. doi:10.1007/s10846-021-01362-w.
- [18] A. Rosinol, M. Abate, Y. Chang, L. Carlone, Kimera: an open-source library for real-time metric-semantic localization and mapping, in: *2020 IEEE International Conference on Robotics and Automation (ICRA)*, 2020, pp. 1689–1696. doi:10.1109/ICRA40945.2020.9196885.
- [19] S. Macenski, I. Jambreci, Slam toolbox: Slam for the dynamic world, *Journal of Open Source Software* 6 (2021). URL: <https://doi.org/10.21105/joss.02783>. doi:10.21105/joss.02783.
- [20] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, W. Burgard, Octomap: an efficient probabilistic 3d mapping framework based on octrees, *Autonomous Robots* 34 (2013) 189–206. URL: <https://doi.org/10.1007/s10514-012-9321-0>. doi:10.1007/s10514-012-9321-0.
- [21] S. Macenski, D. Tsai, M. Feinberg, Spatio-temporal voxel layer: A view on robot perception for the dynamic world, *International Journal of Advanced Robotic Systems* 17 (2020). URL: <https://doi.org/10.1177/1729881420910530>. doi:10.1177/1729881420910530.
- [22] A. akbar Agha-mohammadi, E. Heiden, K. Hausman, G. Sukhatme, Confidence-rich grid mapping, *The International Journal of Robotics Research* 38 (2019) 1352–1374. URL: <https://doi.org/10.1177/0278364919839762>. doi:10.1177/0278364919839762.
- [23] D. V. Lu, D. Hershberger, W. D. Smart, Layered costmaps for context-sensitive navigation, in: *2014 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2014, pp. 709–715. doi:10.1109/IRDS.2014.6942636.
- [24] Z. Fu, A. Kumar, A. Agarwal, H. Qi, J. Malik, D. Pathak, Coupling vision and proprioception for navigation of legged robots, in: *Proceedings of*

- the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022, pp. 17273–17283.
- [25] D. D. Fan, K. Otsu, Y. Kubo, A. Dixit, J. Burdick, A. Akbar Agha-mohammadi, STEP: Stochastic Traversability Evaluation and Planning for Risk-Aware Off-road Navigation, in: *Proceedings of Robotics: Science and Systems*, Virtual, 2021. doi:10.15607/RSS.2021.XVII.021.
 - [26] D. D. Fan, A.-a. Agha-mohammadi, E. A. Theodorou, Learning risk-aware costmaps for traversability in challenging environments, *IEEE Robotics and Automation Letters* 7 (2022) 279–286. doi:10.1109/LRA.2021.3125047.
 - [27] R. Sutton, A. Barto, Reinforcement Learning, second edition: An Introduction, Adaptive Computation and Machine Learning series, MIT Press, 2018. URL: <https://books.google.cz/books?id=5s-MEAAAQBAJ>.
 - [28] T. Haarnoja, A. Zhou, P. Abbeel, S. Levine, Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, in: J. Dy, A. Krause (Eds.), *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, PMLR, 2018, pp. 1861–1870. URL: <https://proceedings.mlr.press/v80/haarnoja18b.html>.
 - [29] A. Kumar, Z. Fu, D. Pathak, J. Malik, RMA: Rapid Motor Adaptation for Legged Robots, in: *Proceedings of Robotics: Science and Systems*, Virtual, 2021. doi:10.15607/RSS.2021.XVII.011.
 - [30] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, P. Abbeel, Domain randomization for transferring deep neural networks from simulation to the real world, in: *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 23–30. doi:10.1109/IROS.2017.8202133.
 - [31] G. Margolis, G. Yang, K. Paigwar, T. Chen, P. Agrawal, Rapid Locomotion via Reinforcement Learning, in: *Proceedings of Robotics: Science and Systems*, New York City, NY, USA, 2022. doi:10.15607/RSS.2022.XVIII.022.
 - [32] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, M. Hutter, Learning quadrupedal locomotion over challenging terrain, *Science Robotics* 5 (2020). URL: <https://www.science.org/doi/abs/10.1126/scirobotics.abc5986>. doi:10.1126/scirobotics.abc5986.
 - [33] T. Miki, J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, M. Hutter, Learning robust perceptive locomotion for quadrupedal robots in the wild, *Science Robotics* 7 (2022). URL: <https://www.science.org/doi/abs/10.1126/scirobotics.abk2822>. doi:10.1126/scirobotics.abk2822.
 - [34] A. Loquercio, A. Kumar, J. Malik, Learning visual locomotion with cross-modal supervision, in: *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 7295–7302. doi:10.1109/ICRA48891.2023.10160760.
 - [35] S. Macenski, S. Singh, F. Martín, J. Ginés, Regulated pure pursuit for robot path tracking, *Autonomous Robots* 47 (2023) 685–694. URL: <https://doi.org/10.1007/s10514-023-10097-6>. doi:10.1007/s10514-023-10097-6. arXiv:1573-7527.
 - [36] R. C. Coulter, Implementation of the Pure Pursuit Path Tracking Algorithm, Technical Report CMU-RI-TR-92-01, Carnegie Mellon University, Pittsburgh, PA, 1992.
 - [37] D. Fox, W. Burgard, S. Thrun, The dynamic window approach to collision avoidance, *IEEE Robotics and Automation Magazine* 4 (1997) 23–33. doi:10.1109/100.580977.
 - [38] K. Rebai, O. Azouaoui, M. Benmami, A. Larabi, Car-like robot navigation at high speed, in: *2007 IEEE International Conference on Robotics and Biomimetics (ROBIO)*, 2007, pp. 2053–2057. doi:10.1109/ROBIO.2007.4522484.
 - [39] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, E. A. Theodorou, Aggressive driving with model predictive path integral control, in: *2016 IEEE International Conference on Robotics and Automation (ICRA)*, 2016, pp. 1433–1440. doi:10.1109/ICRA.2016.7487277.
 - [40] C. Rösmann, F. Hoffmann, T. Bertram, Integrated online trajectory planning and optimization in distinctive topologies, *Robotics and Autonomous Systems* 88 (2017) 142–153. URL: <https://www.sciencedirect.com/science/article/pii/S0921889016300495>. doi:https://doi.org/10.1016/j.robot.2016.11.007.
 - [41] C. Rösmann, F. Hoffmann, T. Bertram, Kinodynamic trajectory optimization and control for car-like robots, in: *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 5681–5686. doi:10.1109/IROS.2017.8206458.
 - [42] D. Dolgov, S. Thrun, M. Montemerlo, J. Diebel, Path planning for autonomous vehicles in unknown semi-structured environments, *The International Journal of Robotics Research* 29 (2010) 485–501. URL: <https://doi.org/10.1177/0278364909359210>. doi:10.1177/0278364909359210.
 - [43] T. Hines, K. Stepanas, F. Talbot, I. Sa, J. Lewis, E. Hernandez, N. Kottege, N. Hudson, Virtual surfaces and attitude aware planning and behaviours for negative obstacle navigation, *IEEE Robotics and Automation Letters* 6 (2021) 4048–4055. doi:10.1109/LRA.2021.3065302.
 - [44] M. Pivtoraiko, R. Knepper, A. Kelly, Optimal, Smooth, Nonholonomic Mobile Robot Motion Planning in State Lattices, Technical Report, 2007.
 - [45] M. Pivtoraiko, A. Kelly, Generating near minimal spanning control sets for constrained motion planning in discrete state spaces, in: *2005 IEEE/RSJ International Conference on Intelligent Robots and Systems*, IEEE, 2005. URL: <https://doi.org/10.1109/2FIROS.2005.1545046>. doi:10.1109/iros.2005.1545046.
 - [46] A. Nash, S. Koenig, C. Tovey, Lazy theta*: Any-angle path planning and path length analysis in 3d, *Proceedings of the AAAI Conference on Artificial Intelligence* 24 (2010) 147–154. URL: <https://ojs.aaai.org/index.php/AAAI/article/view/7566>. doi:10.1609/aaai.v24i1.7566.
 - [47] A. Stentz, The focussed d* algorithm for real-time replanning, in: *Proceedings of the 14th International Joint Conference on Artificial Intelligence - Volume 2, IJCAI'95*, Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 1995, p. 1652–1659.
 - [48] S. Koenig, M. Likhachev, Fast replanning for navigation in unknown terrain, *IEEE Transactions on Robotics* 21 (2005) 354–363. doi:10.1109/TR0.2004.838026.
 - [49] S. Macenski, F. Martín, R. White, J. G. Clavero, The marathon 2: A navigation system, in: *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020, pp. 2718–2725. doi:10.1109/IROS45743.2020.9341207.
 - [50] M. Mateas, A. Stern, A behavior language for story-based believable agents, *IEEE Intelligent Systems* 17 (2002) 39–47. doi:10.1109/MIS.2002.1024751.
 - [51] M. Nicolau, D. Perez-Liebana, M. O'Neill, A. Brabazon, Evolutionary behavior tree approaches for navigating platform games, *IEEE Transactions on Computational Intelligence and AI in Games* 9 (2017) 227–238. doi:10.1109/TCIAIG.2016.2543661.
 - [52] A. Marzinotto, M. Colledanchise, C. Smith, P. Ögren, Towards a unified behavior trees framework for robot control, in: *2014 IEEE International Conference on Robotics and Automation (ICRA)*, 2014, pp. 5420–5427. doi:10.1109/ICRA.2014.6907656.
 - [53] J. A. Bagnell, F. Cavalcanti, L. Cui, T. Galluzzo, M. Hebert, M. Kazemi, M. Klingensmith, J. Libby, T. Y. Liu, N. Pollard, M. Pivtoraiko, J.-S. Valois, R. Zhu, An integrated system for autonomous robotics manipulation, in: *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012, pp. 2955–2962. doi:10.1109/IROS.2012.6385888.
 - [54] M. Colledanchise, P. Ögren, Behavior Trees in Robotics and AI, Chapman & Hall/CRC Artificial Intelligence and Robotics Series, CRC Press, 2019. URL: https://books.google.cz/books?hl=sk&lr=&id=YVOWDwAAQBAJ&oi=fnd&pg=PP1&ots=hyJAh5RbjM&sig=sGzNq3Y19WjE4kND4T9oAUMJII4&redir_esc=y#v=onepage&q&f=false.
 - [55] X. Cheng, A. Kumar, D. Pathak, Legs as manipulator: Pushing quadrupedal agility beyond locomotion, in: *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 5106–5112. doi:10.1109/ICRA48891.2023.10161470.
 - [56] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, 2017. arXiv:1707.06347.
 - [57] S.-K. Kim, A. Bouman, G. Salhotra, D. D. Fan, K. Otsu, J. Burdick, A.-a. Agha-mohammadi, Pilgrim: Hierarchical value learning for large-scale exploration in unknown environments, *Proceedings of the International Conference on Automated Planning and Scheduling* 31 (2021) 652–662. URL: <https://ojs.aaai.org/index.php/ICAPS/article/view/16014>. doi:10.1609/icaps.v31i1.16014.
 - [58] E. Marder-Eppstein, E. Berger, T. Foote, B. Gerkey, K. Konolige, The office marathon: Robust navigation in an indoor office environment, in: *2010 IEEE International Conference on Robotics and Automation*, 2010, pp. 300–307. doi:10.1109/ROBOT.2010.5509725.

- [59] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, Robot operating system 2: Design, architecture, and uses in the wild, *Science Robotics* 7 (2022). URL: <https://www.science.org/doi/abs/10.1126/scirobotics.abm6074>. doi:10.1126/scirobotics.abm6074.
- [60] O. N. LLC, Setting up navigation plugins, 2023. URL: https://navigation.ros.org/setup_guides/algorithm/select_algorithm.html, accessed: 2023-07-28.
- [61] O. N. LLC, Tuning guide, 2023. URL: <https://navigation.ros.org/tuning/index.html>, accessed: 2023-07-28.
- [62] O. S. R. F. Inc, teb_local_planner, 2023. URL: http://wiki.ros.org/teb_local_planner?distro=galactic, accessed: 2023-07-28.